Ankara Üniversitesi
Yapay Zekâ ve Veri Mühendisliği Bölümü
YZM327 · Yazılım Mühendisliği için Yapay Zekâ · 2026–2027 Güz



Kurultay

Oylamayla yönetilen, sansürsüz bir forumun Cosmic Python mimarisi ve GoF desenleriyle yeniden yapılandırılması

Kurultay, eski Türk devletlerinde kararın birlikte konuşularak alındığı meclistir. Bu forumda da hiçbir değişiklik tek bir kişinin elinde değildir: konu açmak, düzenlemek, gizlemek ve kuralları değiştirmek, o konuyu ilgilendiren üyelerin oyuna sunulan birer tekliftir.

Belge	**Teknik Rapor · Ödev 1 (ikinci sürüm)**
Ders yürütücüsü	**Doç. Dr. Savaş Takan**
Hazırlayan	**Seyyid A. Bozkurt · 24290381**
Kaynak kod	**src/kurultay · Python 3.12 · FastAPI · SQLAlchemy 2 · SQLite**
Tarih	**27 Ekim 2026**

Özet

Kurultay, gündemi ve kuralları üyelerin oylamasıyla belirlenen bir tartışma forumudur. Herkes konu açabilir, alt başlık, düzenleme ya da gizleme önerebilir; ancak hiçbir işlem doğrudan gerçekleşmez, her biri bir *teklif* olarak oylamaya sunulur. Bir teklife yalnızca konunun ontolojisiyle eşleşen üyeler ve yapay zekâ (YZ) uzman ajanları oy verebilir. Tartışmalar fiziksel olarak silinmez; oylamayla gizlenen içerik maskelenir ve her olay SHA-256 özet zinciriyle bağlı, üç düğümde çoğaltılan bir deftere yazılır. Tek içerik yasağı tehdittir: hakaret dahil her görüş, siyasi yelpazenin neresinden gelirse gelsin, yayında kalır.

İlk sürüm (16 Ekim 2026, Next.js) işlevsel olarak tamamdı, ancak iş kuralları rotalara ve arayüz bileşenlerine dağılmıştı ve ders kurallarıyla (katmanlı mimari, bağımlılığın tersine çevrilmesi, Repository/Unit of Work, Message Bus) ölçülebilir değildi. Bu rapor, sistemin Python ile *Architecture Patterns with Python* (Cosmic Python) kitabının dört katmanlı yapısına göre baştan yazılmış ikinci sürümünü anlatır. Alan katmanı hiçbir çerçeveye bağımlı değildir ve bu kural testle denetlenir. Bütün durum değişiklikleri komut ve alan olayı olarak tek bir Message Bus'tan geçer. Ödevde istenen sekiz GoF deseni (Composite, State, Strategy, Adapter, Factory/Registry, Chain of Responsibility, Protection Proxy, Observer) adlarıyla ve gerekçeleriyle uygulanmıştır. Her desen için “neden bu, neden alternatifi değil” sorusu ders notlarındaki seçim rehberine göre yanıtlanmıştır.

Rapor yalnızca yapılanı anlatmakla kalmaz, ilk taslağa bir değerlendirici gözüyle bakıldığında beğenilmeyen üç yeri ve nasıl düzeltildiklerini de gösterir. Birincisi, karesel oylamada kredinin her teklif için ayrı verilmesiydi; bu, rasyonel bir seçmen için yöntemi “her şeye 3 oy” düzeyine indiriyordu. İkincisi, ölçümü olmayan bir tehdit kuralıydı. Üçüncüsü, süreç düzeyinde gizli global önbelleklerdi. Hafta 2'nin problem çerçeveleme ilkelerine uygun olarak her YZ bileşeni için önce “YZ gerekli mi?” sorusu sorulmuştur. Tehdit kuralı, etiketli bir sette temel çizgi merdiveniyle ölçülmüştür: F0.5 = 0,83 (naif anahtar kelime 0,44, rastgele 0,28).

149 pytest testi, 0 başarısız (118 birim, 20 entegrasyon, 11 uçtan uca); 32 sn	**0** ters yönlü bağımlılık; beş mimari kural testle korunur	**0,83** tehdit kuralında F0.5 (naif yöntem 0,44); hata maliyeti 45 → 10	**0 ₺** ücretli servis yok; YZ ajanları kural tabanlı ya da yerel Ollama
3000 üyelik simülasyon; 25 konu, 43 teklif, ≈9 700 oy, ≈13 000 defter kaydı	**4,6 ms** bir oyun uçtan uca işlenmesi (medyan, 1 013 kişilik kütükte)	**14** Mermaid UML diyagramı: 9 sınıf/durum, 5 sıralama	**8** GoF deseni; ayrıca Repository, UoW, Message Bus, CQRS

Anahtar kelimeler: Cosmic Python, temiz mimari, bağımlılığın tersine çevrilmesi, Repository, Unit of Work, Message Bus, GoF tasarım desenleri, karesel oylama, azınlık koruması, ontoloji, hash zinciri, problem çerçeveleme, temel çizgi.

İçindekiler

1	Giriş	4
2	Gereksinimler ve izlenebilirlik	5
3	Problem çerçeveleme: YZ nerede gerekli?	6
4	Mimari: dört katman ve bağımlılığın tersine çevrilmesi	8
5	Alan modeli ve kişisel verinin gizliliği	10
6	Tasarım desenleri: neden bu, neden diğeri değil	12
7	Yönetişim: her şey bir teklif	18
8	Azınlığın korunması	20
9	Değiştirilemez dağıtık defter	21
10	YZ uzman ajanları	23
11	Ölçüm ve değerlendirme	24
12	Davranış: sıralama diyagramları	26
13	Test stratejisi	30
14	Ölçek: 3000 üyelik simülasyon	31
15	Arayüz	32
16	Ön-otopsi, sınırlılıklar ve sonraki adımlar	34
17	Sonuç	35
◆	YZ kullanım beyanı · Kaynakça	35
A	Ek A: Kurulum, çalıştırma ve yeniden üretim	36

Şekiller

1	Katmanlar ve bağımlılığın tersine çevrilmesi (sınıf diyagramı)	9
2	Alan modeli: kullanıcı, ontoloji, teklif ve seçmen kütüğü (sınıf diyagramı)	11
3	Profil görüntüleme ve PII koruması (sıralama diyagramı)	12
4	Composite ve State (sınıf diyagramı)	14
5	Konu yaşam döngüsü (durum diyagramı)	14
6	Strategy: oylama algoritmaları ve Registry (sınıf diyagramı)	15
7	Chain of Responsibility ve Protection Proxy (sınıf diyagramı)	16
8	Observer, Adapter ve Factory/Registry (sınıf diyagramı)	17
9	Repository, Unit of Work ve Message Bus (sınıf diyagramı)	18
10	Canlı oylamanın sonucu	21
11	Defter sayfası: son kayıtlar, her kaydın önceki ve kendi özeti, düğümlerin uzlaşma raporu ve kurcalama gösterimi	22
12	Dağıtık defter: hash zinciri, düğüm kopyaları ve çoğunluk uzlaşması (sınıf diyagramı)	23
13	Konu önerisi (sıralama diyagramı)	27
14	Oy verme ve sonuçlanma (sıralama diyagramı)	28
15	Mesaj gizleme (sıralama diyagramı)	29
16	Defter kaydı, çoğaltma ve doğrulama (sıralama diyagramı)	29
17	Gündem (açık tema)	32
18	Aynı sayfa, koyu tema	32
19	Bir tartışma	32
20	Bir teklif sayfası (karesel oylama)	33
21	Hesap sahibinin kendi profili: ad, doğum tarihi ve adres yalnızca ona gösterilir	33
22	Aynı profil, başka birinin gözünden: yalnızca takma ad, ilgi alanları ve katkılar	33

1 Giriş

1.1 Problem

Çevrimiçi tartışma platformlarında iki sorun birbirini besler. Birincisi moderasyonun keyfiliğidir: bir içeriğin kalıp kalmayacağına çoğu zaman birkaç kişi karar verir ve bu karar geriye iz bırakmadan uygulanır. İkincisi çoğunluk tiranlığıdır: kararlar oylamayla alınsa bile, konuyu az önemseyen kalabalık bir grup, konuyu çok önemseyen küçük bir grubu her seferinde ezebilir. Kurultay bu iki sorunu birlikte ele alır. Hiçbir işlem doğrudan yapılmaz, her değişiklik oylanır. Oylama algoritması azınlığın yoğun itirazını sayıya dökebilir. Oy hakkı, konuyla ilgisi olanlarla sınırlıdır. Verilen her karar silinemez bir deftere yazılır.

1.2 Bu sürümde ne değişti?

İlk sürüm aynı fikri TypeScript ile gerçekleştirmişti ve özellik olarak eksiksizdi, fakat bir değerlendirici gözüyle bakıldığında iş kurallarının nerede yaşadığı belli değildi. Oylama eşiği bir API rotasında, uygunluk denetimi bir React bileşeninde, defter yazımı veritabanı yardımcı fonksiyonlarında duruyordu. Bu dağınıklık, ödevin asıl ölçütü olan mimari kuralların (alanın altyapıdan habersiz olması, veri erişiminin soyutlanması, durum değişikliklerinin tek kapıdan geçmesi) test edilmesini imkânsız kılıyordu. İkinci sürüm bu yüzden yamayla değil, yeniden yazımla yapıldı. İlk sürümün değerli parçaları korundu.

KONU	İLK SÜRÜM (16 EKİM)	İKİNCİ SÜRÜM (BU RAPOR)
Dil ve çatı	TypeScript, Next.js 16, React	Python 3.12, FastAPI + Jinja2 (sunucu taraflı HTML)
İş kurallarının yeri	Rotalar, bileşenler, yardımcılar	`domain/` (saf Python, yalnızca standart kütüphane)
Veri erişimi	Doğrudan SQL çağrıları	Repository + Unit of Work; SQLAlchemy Core ile açık veri eşleyici
Durum değişikliği	Her rota kendi işini yapıyordu	Komut → Message Bus → işleyici → alan olayları → gözlemciler
Test	15 birim testi (yalnızca karar motoru)	149 test; Fake bileşenlerle, veritabanı olmadan çalışan birim testleri
Korunanlar	Tehdit kuralı ve 60 cümlelik etiketli set, 11 ana + 8 alt kavramlık ontoloji, hash zinciri fikri, otağ ve kilim esinli arayüz dili	

1.3 Raporun okunuşu

Her bölüm aynı düzeni izler: önce problem, sonra seçilen çözüm, sonra seçilmeyen alternatifler ve en sonda “Nerede?” kutusunda ilgili dosyalar ile o davranışı kanıtlayan testler. Kırmızı kenarlı **öz eleştiri** kutuları, ilk taslakta beğenilmeyip düzeltilen yerleri gösterir. Bütün sayılar depodaki komutlarla yeniden üretilebilir (Ek A). Diyagramların Mermaid kaynakları `docs/diyagramlar/` klasöründedir ve uygulamanın “Mimari” sayfasında da görüntülenir.

Kaynak kodun haritası

`src/kurultay/domain` (17 modül, ≈2 200 satır) · `service_layer` (9 modül, ≈1 600) · `adapters` (10 modül, ≈1 350) · `entrypoints` (web + demo, ≈1 950) · `evaluation` (ölçüm, ≈350) · `tests` (≈1 900 satır). Bileşimin kökü `bootstrap.py`'dir.

2 Gereksinimler ve izlenebilirlik

Ödev metnindeki her isteğin hangi kod parçasıyla karşılandığı ve hangi testle kanıtlandığı aşağıdaki matristedir. Bir istek ancak bir testle doğrulanıyorsa "karşılandı" sayılmıştır.

İSTEK	KARŞILIK	KOD · TEST
Dört katman; domain ORM, veritabanı ve dış API'den habersiz	Domain yalnızca standart kütüphaneyi içe aktarır; somut sınıflar `bootstrap.py`'de bağlanır	`test_architecture.py` (6 test)
Repository (Abstract + Fake)	7 soyut repository; her birinin SQLAlchemy ve Fake gerçeklemesi; silme metodu yok	`adapters/repository.py`, `tests/fakes.py`
Context manager tabanlı Unit of Work, olay yayımı	`with uow:` atomik commit/rollback; aggregate olaylarını toplar	`test_uncommitted_work_is_rolled_back`
Command + Domain Event + Message Bus	13 komut, 15 alan olayı; komutun tek, olayın çok işleyicisi	`test_event_handlers_can_queue_follow_up_commands`
Composite: konu ağacı	`TopicComponent` ← `Topic` / `Message`	`test_topic_composite_and_state.py`
State: Önerildi → Oylamada → Aktif → Silinme Oylamasında → Arşivlendi/Gizlendi	7 durum sınıfı; geçersiz geçiş `InvalidTransition`	aynı dosya
Strategy: karesel, uzman ağırlıklı, itibar tabanlı	`VotingStrategy` (`typing.Protocol`) + basit çoğunluk temel çizgisi	`test_voting_and_eligibility.py`
LLM'ler Adapter; seçim Factory/Registry	`OllamaExpertModel`, `RuleBasedExpertModel`, `ExpertModelRegistry`	`test_registry_picks_model_by_key…`
Chain of Responsibility; Protection Proxy	7 halkalı oy hakkı zinciri; `AuthorizationProxy`, `ProfileAccessProxy`	`test_authorization_proxy…`, `test_profile_proxy…`
Observer: oylama başlayınca üyelere ve ajanlara bildirim	`VotingNotificationCenter` + 2 gözlemci	`test_voting_started_notifies…`

İSTEK	KARŞILIK	KOD · TEST
PII: ad, soyad, doğum tarihi, adres, şifre saklanır; yalnızca takma ad görünür	`PersonalInfo` değer nesnesi; `PublicProfile`'da kişisel alan yok	`test_register_user_keeps_pii…`, e2e profil testi
Her şey bir teklif	8 teklif türü; doğrudan değişiklik yapan komut yok	`test_service_layer.py` (28 test)
Yalnızca ontolojisi eşleşenler oy verir	Ontoloji komşuluğu + oylama açılışında dondurulan seçmen kütüğü	`test_uninterested_member_cannot_vote`, `test_members_who_join_after…`
Append-only; gizlenen içerik defterde kalır	Maskeleme; SQLite tetikleyicileri UPDATE/DELETE'i reddeder; hash zinciri	`test_database_triggers_forbid…`, `test_message_hiding_masks…`
Dağıtık defter	3 düğüm (JSON Lines); çoğunluk uzlaşması, kurcalama tespiti	`test_json_lines_nodes_detect_tampering`
pytest + Fake bileşenler	Birim testleri veritabanı ve ağ olmadan çalışır	118 birim testi
UML sınıf ve sıralama diyagramları (Mermaid)	9 sınıf/durum + 5 sıralama diyagramı	`docs/diyagramlar/*.mmd`
Tip belirteçleri, `@dataclass(frozen=True)`	Bütün değer nesneleri, komutlar ve olaylar dondurulmuş	`mypy src`: 52 modülde 0 hata

3 Problem çerçeveleme: YZ nerede gerekli?

Ders notlarının (Hafta 2) ilk sorusu, bir sorunun yapay zekâ gerektirip gerektirmediğidir. Karar ağacı soldan sağa okunur: iş etik ve yasal mı, basit bir kural işi görüyor mu, veri var mı, hatanın bedeli yönetilebilir mi? İlk "dur" kutusunda proje ya biter ya da biçim değiştirir. Kurultay'da YZ'nin değebileceği dört karar noktası bu ağaçtan geçirilmiştir.

KARAR NOKTASI	AĞAÇTA NEREDE DURDU?	SEÇİLEN ÇÖZÜM	GEREKÇE
Tehdit denetimi (mesaj yayımlanabilir mi?)	"Basit bir kural işi görüyor mu?" → **evet** (ölçüldü, Bölüm 11)	Kural: açık kalıp ya da 1. tekil şiddet fiili + hedef zamiri	Açıklanabilir, anlık, deterministik; engellenen kişiye hangi cümlenin neden engellendiği gösterilebilir. Kural, ileride bir modelin yenmesi gereken temel çizgidir.
Kavram önerisi (konu hangi başlığa ait?)	Kural yeterli; üstelik karar vermiyor, yalnızca not düşüyor	Anahtar kelime örtüşmesi (`Ontology.suggest`)	Yanlış öneri zararsızdır: kavramı öneren kişi seçer, oylayanlar görür.
Oylamanın sonucu	"Etik ve yasal mı?" → YZ'nin karar vermesi **hayır**	Karar insanlarındır	Bir topluluğun kuralını bir modelin belirlemesi meşruiyet sorunudur. YZ yalnızca oyu yönetmelikteki oranla sayılan bir seçmen olabilir; yönetmelikle tamamen kapatılabilir.

KARAR NOKTASI	AĞAÇTA NEREDE DURDU?	SEÇİLEN ÇÖZÜM	GEREKÇE
YZ ajanının değerlendirmesi (teklif hakkında gerekçeli görüş)	Veri yok ama BDM veri olmadan başlayabilir; hata bedeli düşük (oy ağırlığı %50)	Yerel BDM (Ollama), kapalıysa kural tabanlı model (Cascade)	Ücretsiz ve yerel çalışır. Model yanıtı ayrıştırılamazsa ya da sunucu kapalıysa yedek model devreye girer; sistem hiçbir zaman bir dış API'ye bağımlı olmaz.

3.1 Tek sayfalık problem kanvası: tehdit denetimi

Ders notlarındaki sekiz kutulu kanvas (Dorard'ın *Machine Learning Canvas*'ından uyarlanmış) Kurultay'ın tek engelleyici YZ bileşenine uygulanmıştır.

1. İş hedefi ve değer	Tehdit dışında hiçbir görüşü engellemeden üyeleri gerçek tehditten korumak. İş metriği: haksız engel (sansür) sayısı.
2. Karar ve eylem	Her mesaj, konu ve teklif metni gönderilirken: yayımla ya da tehdit içeren cümleyi göstererek geri çevir. Günde yüzlerce kez, yanıt anlık olmalı.
3. ML görevi	Metin → {tehdit, tehdit değil}; ikili sınıflandırma. Etiket tanımı: belirli bir kişiye ya da gruba yöneltilmiş ciddi şiddet niyeti beyanı (ABD hukukundaki *true threat*). Hakaret, mecaz, haber dili ve başkasının tehdidini aktarmak tehdit değildir.
4. Veri	60 cümlelik etiketli set (20 tehdit, 40 tehdit değil); kuralın kaçıracağı bilinen örtük tehditler ve yanlış alarm vereceği bilinen mecazlar bilerek eklendi. Kişisel veri yok.
5. Metrikler	Model metriği: **F0.5** (kesinliğe iki kat ağırlık). Koruyucu metrik: duyarlılık ≥ 0,6. Hata maliyeti: YP = 2, YN = 1.
6. Temel çizgi	Rastgele (sınıf oranıyla), çoğunluk sınıfı, naif anahtar kelime (11 şiddet kökü). Sonuçlar Bölüm 11'de.
7. Kısıtlar	Ücretsiz, çevrimdışı, GPU'suz; milisaniyeler içinde yanıt; Türkçe büyük/küçük harf tuzağı (`"I".lower()` → `"i"`).
8. Risk ve başarı	Yayın ölçütü: F0.5 naif yöntemi açık farkla geçmeli ve kesinlik ≥ 0,8 olmalı. Risk: kural yeni kalıpları kaçırır (Bölüm 16'daki ön-otopsi).

3.2 Hangi hata daha pahalı?

Ders notu, eşiğin bir iş kararı olduğunu vurgular. Kurultay'da iki hata türünün bedeli simetrik değildir. **Yanlış pozitif** (masum bir sözün engellenmesi) yazarın sesini o anda keser ve itiraz edebileceği bir oylama yoktur, çünkü mesaj hiç yayımlanmamıştır. **Yanlış negatif** (kaçan bir tehdit) ise sistemde ikinci bir savunma hattına takılır: herhangi bir üye mesajın gizlenmesini teklif edebilir ve topluluk karesel oylamayla mesajı maskeler. Bu yüzden ana metrik F0.5'tir ve maliyet fonksiyonu yanlış pozitifi iki kat sayar. Kural, belirsiz durumda yayımlamayı seçer.

Nerede?

`domain/text.py` (`find_threat` , Türkçe ön işleme hattı) · `domain/content_policy.py` (içerik zinciri: boş metin → tehdit → ontoloji notu) · `evaluation/` (etiketli set, metrikler, temel çizgi merdiveni) · `tests/unit/test_evaluation.py`

4 Mimari: dört katman ve bağımlılığın tersine çevrilmesi

Sistem, Cosmic Python'un önerdiği dört katmana ayrılmıştır ve bağımlılıklar yalnızca içe doğru akar. Alan modeli (domain) en içtedir ve hiçbir şeyi tanımaz. Servis katmanı alan modelini ve kendi tanımladığı *portları* (soyut arayüzleri) bilir. Altyapı (adapters) bu portları gerçekler. Giriş noktaları (entrypoints) HTTP isteğini komuta çevirir. Somut sınıfların hangi soyutlamaya bağlanacağına tek bir yerde, `bootstrap.py`'deki bileşim kökünde (composition root) karar verilir.

KATMAN	SORUMLULUK	BAĞIMLI OLABİLECEĞİ
Domain	Aggregate'ler, değer nesneleri, desenler (Composite, State, Strategy, Chain), ontoloji, hash zinciri, komut ve olay tanımları	Yalnızca Python standart kütüphanesi
Service layer	Komut ve olay işleyicileri, Message Bus, Unit of Work sözleşmesi, Observer merkezi, YZ ajanı fabrikası, Protection Proxy'ler, CQRS okuma görünümleri, portlar	Domain; repository *soyutlamaları*
Adapters	SQLAlchemy Core şeması ve repository'ler, YZ model adaptörleri, defter düğümleri, PBKDF2, saat, oturum imzası	Domain, portlar, üçüncü parti kütüphaneler
Entrypoints	FastAPI + Jinja2 web uygulaması, demo simülasyonu, canlı oylama gösterimi	Hepsi (yalnızca bileşim için)

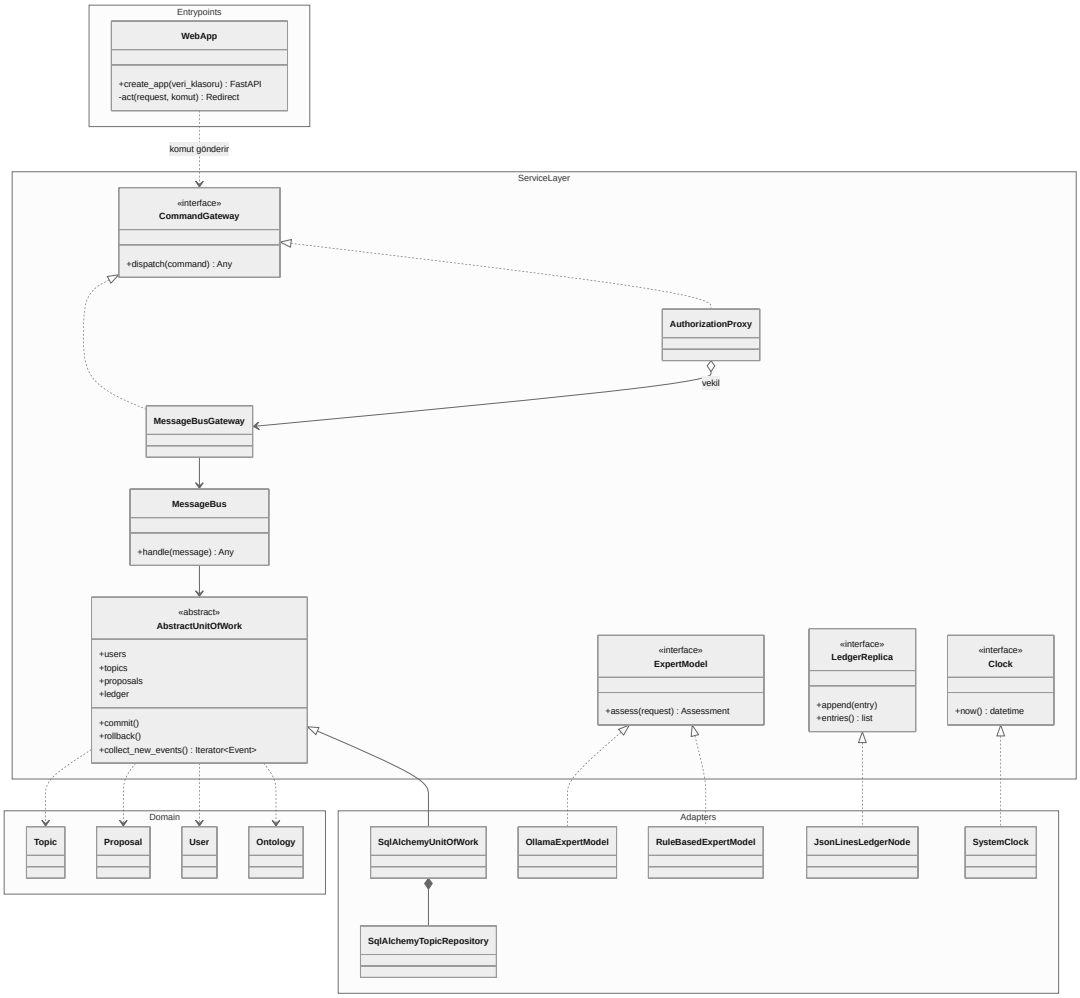
4.1 Portlar ve bağımlılık enjeksiyonu

Servis katmanı dört portu `typing.Protocol` ile tanımlar: `Clock`, `PasswordHasher`, `ExpertModel` ve `LedgerReplica`. İşleyiciler bağımlılıklarını parametre olarak bildirir; `bootstrap` her işleyicinin imzasına bakıp yalnızca istediği bağımlılıkları verir. Bu, Cosmic Python'un 13. bölümündeki elle yapılan bağımlılık enjeksiyonudur. Bir DI çerçevesi (ders notlarındaki Hydra örneği gibi) bu ölçekte gereksiz bir bağımlılık olurdu.

```
# bootstrap.py — bileşim kökü: somut sınıflar yalnızca burada seçilir
def inject(handler, dependencies):
    params = inspect.signature(handler).parameters
    deps = {name: dep for name, dep in dependencies.items() if name in params}
    return lambda message: handler(message, **deps)

# service_layer/handlers.py — işleyici yalnızca soyutlamaları bilir
def cast_vote(cmd: commands.CastVote, uow: AbstractUnitOfWork, clock: Clock) -> None: ...
```

Aynı mekanizma testlerde de kullanılır. `FakeUnitOfWork`, `ManualClock`, `FakePasswordHasher` ve `FakeExpertModel` enjekte edildiğinde bütün servis katmanı veritabanı ve ağ olmadan, milisaniyeler içinde çalışır.



Şekil 1. Katmanlar ve bağımlılığın tersine çevrilmesi (sınıf diyagramı). Adapters, servis katmanındaki portları gerçekler; ok yönü "bilir" ilişkisidir ve hiçbir ok domain'den dışarı çıkmaz.

4.2 Mimari kuralların kendisi de test edilir

Bir kural yalnızca belgede yazıyorsa, ilk aceleli değişiklikte bozulur. `tests/unit/test_architecture.py` kaynak kodu `ast` ile tarar ve beş kuralı denetler: (1) domain yalnızca kendini ve standart kütüphaneyi içe aktarır; (2) işleyiciler somut repository'leri ve SQLAlchemy'yi tanımaz; (3) servis katmanı ve adapters, entrypoints'i içe aktarmaz; (4) hiçbir repository'de adı *delete* ya da *remove* içeren bir metot yoktur (append-only); (5) servis katmanında ve adapters'ta modül düzeyinde değiştirilebilir bir koleksiyon (gizli global durum) yoktur.

> **Öz eleştiri: gizli global önbellek**
>
> İlk taslakta 3000 üyelik demo hızlansın diye seçmen kütükleri ve oylar `repository.py` 'de modül düzeyindeki iki sözlükte önbelleğe alınıyordu. Ders notlarındaki anti-desen listesinde "Singleton bağımlılığı: gizli global durum" olarak geçen sorun tam olarak buydu. Önbellek bütün süreçte paylaşılıyordu. Satır numarasıyla (rowid) artımlı okuma yaptığı için, aynı süreçte iki farklı veritabanı açıldığında birinin satırları diğerinin önbelleğine karışabilirdi.
>
> **Düzeltme:** Önbellek `AppendOnlyReadCache` adlı açık bir nesneye taşındı. `bootstrap` her veritabanı motoru için bir tane üretip Unit of Work'e enjekte eder. Defter doğrulamasının sonucu da aynı şekilde web uygulamasına ait bir nesnede tutulur. Kuralın geri gelmemesi için 5 numaralı mimari test eklendi.

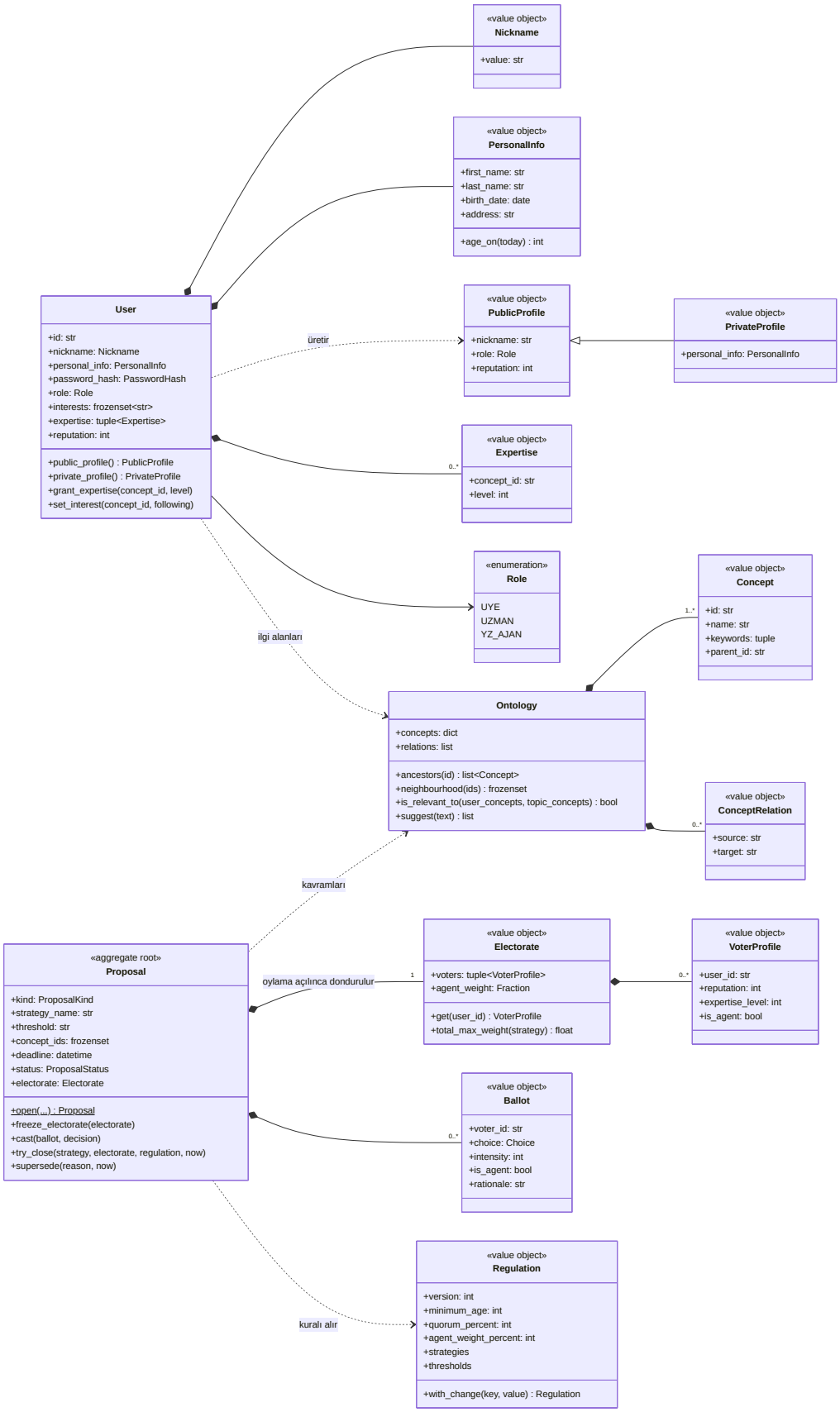
4.3 Cosmic Python ile eşleme

KİTAP BÖLÜMÜ	KURULTAY'DAKİ KARŞILIĞI
1 · Alan modellemesi	`User`, `Topic`, `Proposal` aggregate'leri; `Nickname`, `PersonalInfo`, `Ballot`, `Electorate` değer nesneleri
2 · Repository	`AbstractRepository[T]` (generic) ve 7 alt sözleşme; `seen` kümesiyle olay toplama
3 · Bağlaşım ve soyutlamalar	Portlar; YZ modelinin ve saatin dışarıdan verilmesi
4 · Servis katmanı	`handlers.py`: her kullanım durumu tek bir fonksiyon
5 · Yüksek ve düşük viteste TDD	Testlerin çoğu servis katmanında (Fake'lerle); alan testleri kenar durumlar için
6 · Unit of Work	`AbstractUnitOfWork` (context manager), `SqlAlchemyUnitOfWork`, `FakeUnitOfWork`
7 · Aggregate ve tutarlılık sınırları	Konu ağacı tek aggregate (kök `Topic`); oylar `Proposal`'ın içinde; aggregate'ler arası referans yalnızca kimlikle
8–9 · Olaylar ve Message Bus	`events.py` (15 olay); olay işleyicileri izole (biri patlarsa diğerleri çalışır)
10 · Komutlar	`commands.py` (13 komut); komut hatası çağırana iletilir, olay hatası günlüğe yazılır
11 · Olay güdümlü mimari	Gözlemcilerin döndürdüğü komutlar (YZ ajanının oyu) kuyruğun sonuna eklenir
12 · CQRS	`views.py`: okuma tarafı yalnızca okur, ekran için düz veri nesneleri üretir
13 · Bağımlılık enjeksiyonu	`bootstrap.py` ve `inject`

5 Alan modeli ve kişisel verinin gizliliği

Alan modelinde üç aggregate vardır. Her biri kendi tutarlılık sınırını korur ve diğerlerine yalnızca kimlikle başvurur.

- **`User`**: takma ad, kişisel bilgiler, şifre özeti, rol, ilgi alanları (ontoloji kavramları), uzmanlıklar ve itibar. Uzmanlık ancak bir `GRANT_EXPERTISE` teklifi kabul edilirse verilir.
- **`Topic`**: konu ağacının köküdür. Alt konular ve mesajlar bu aggregate'in içindedir (Composite). Düzenlemeler eski sürümün üzerine yazılmaz, `TopicRevision` olarak eklenir. Gizleme bir `Mask` değer nesnesidir.
- **`Proposal`**: türü, stratejisi, eşiği, son tarihi, dondurulmuş seçmen kütüğü (`Electorate`) ve oylarıyla bir teklif. Oy ancak yetki zincirinden olumlu karar geldiyse kabul edilir; sonuç kesinleşince teklif kendini kapatır.

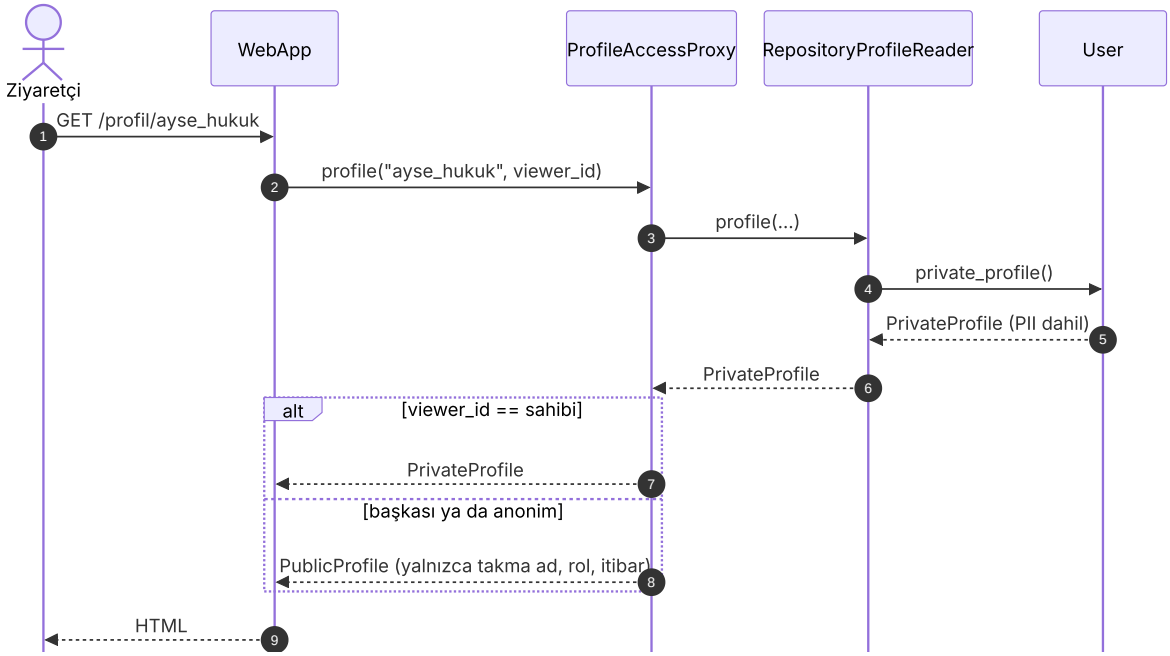


Şekil 2. Alan modeli: kullanıcı, ontoloji, teklif ve seçmen kütüğü (sınıf diyagramı). Değer nesneleri `@dataclass(frozen=True)`'dur. `Electorate`, oylama açıldığında dondurulan seçmen listesidir.

5.1 Kişisel veri: saklanır ama görünmez

Ödev, her kullanıcının adını, soyadını, doğum tarihini, adresini ve şifresini saklamayı, ama arayüzde yalnızca takma adı göstermeyi ister. Bu kural üç katmanda, birbirinden bağımsız olarak uygulanır. Böylece bir katmandaki hata tek başına veri sızdırmaz:

1. **Tip düzeyi:** `PersonalInfo` dondurulmuş bir değer nesnesidir ve `__repr__`'ı içeriği yazmaz; kişisel veri yanlışlıkla bir günlüğe düşmez. `PublicProfile` sınıfında kişisel veri alanı *hiç yoktur*, dolayısıyla bir şablon onu gösteremez. `PrivateProfile` ondan türetilir ve kişisel alanları ekler.
2. **Erişim düzeyi:** `ProfileAccessProxy` (Protection Proxy) tam profili yalnızca hesabın sahibine verir; başka herkes `PublicProfile` alır (Şekil 3).
3. **Kayıt düzeyi:** Deftere kişisel veri ve mesaj metni yazılmaz, yalnızca metnin SHA-256 özeti (`ContentHash`) yazılır. Şifreler PBKDF2-SHA256 ile, kullanıcıya özgü tuzla saklanır.



Şekil 3. Profil görüntüleme ve PII koruması (sıralama diyagramı). Proxy, isteyenin kimliğini profil sahibininkiyle karşılaştırır; sahibi değilse `PublicProfile` döner. `PublicProfile`'da kişisel veri alanı olmadığı için şablon onu yanlışlıkla gösteremez.

---

6 Tasarım desenleri: neden bu, neden diğeri değil

Ders notlarındaki seçim rehberi, bir desenin ancak tanımlı bir soruna yanıt olarak kullanılmasını ister: “desen, ihtiyaç ortaya çıktığında uygulanmalı” (YAGNI, KISS). Aşağıdaki tablo her deseni, rehberdeki “karşılaştığınız durum” sütunuyla eşler. Ardından her desen için sık karıştırıldığı kardeşinin neden seçilmediği tartışılır.

DESEN	KURULTAY'DAKİ DURUM (REHBERİN DİLİNDE)	SINIFLAR
Composite	Parça–bütün ağacı tek arayüzle gezilmeli	`TopicComponent`, `Topic`, `Message`
State	Uzun `if durum == …` zincirleri	`TopicState` ve 7 alt sınıf
Strategy	Aynı işi yapan birden fazla algoritma, çalışma anında seçim	`VotingStrategy` + 4 gerçekleme

DESEN	KURULTAY'DAKİ DURUM (REHBERİN DİLİNDE)	SINIFLAR
Factory / Registry	Ada ya da yapılandırmaya göre nesne üretmek	`STRATEGIES`, `ExpertModelRegistry`
Adapter	Harici kütüphanenin arayüzü benimkine uymuyor	`OllamaExpertModel`, `RuleBasedExpertModel`
Chain of Responsibility	Ardışık doğrulama / filtreleme kuralları	`EligibilityHandler` (7 halka), `ContentCheck`
Protection Proxy	Gerçek nesneye erişimi denetlemek	`AuthorizationProxy`, `ProfileAccessProxy`
Observer	Olay olduğunda birden çok bileşen haberdar olmalı	`VotingNotificationCenter` + 2 gözlemci
Command	İşlemleri saklamak, kuyruğa almak	`commands.py`; Message Bus kuyruğu
Memento (sürüm)	Önceki hâli kaybetmeden değiştirmek	`TopicRevision`, yönetmelik sürümleri

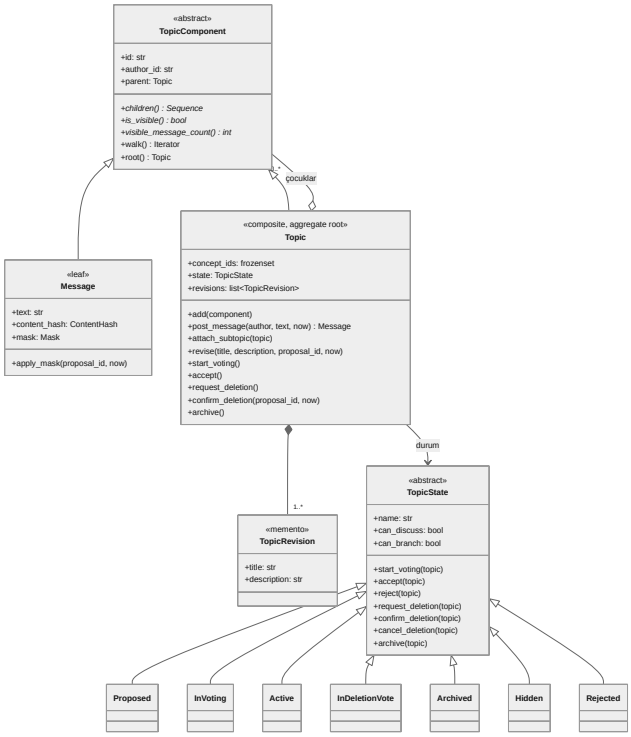
6.1 Composite ve State: konu ağacı ve yaşam döngüsü

Bir konu, alt konular ve mesajlardan oluşan bir ağaçtır. Arayüzün “bu dalda kaç görünür mesaj var”, “bu düğüm gizli mi” gibi soruları ağacın her düzeyinde aynı biçimde sorabilmesi gerekir. `TopicComponent` soyut sınıfı bu ortak arayüzü tanımlar. `Topic` bileşik (çocukları olan), `Message` yapraktır. `visible_message_count()` gibi bir işlem bileşikte çocuklara özyinelemeli olarak dağıtılır. Gizleme de aynı arayüzden geçer: maskeli bir düğüm ağaçta kalır, yalnızca görünürlüğü değişir.

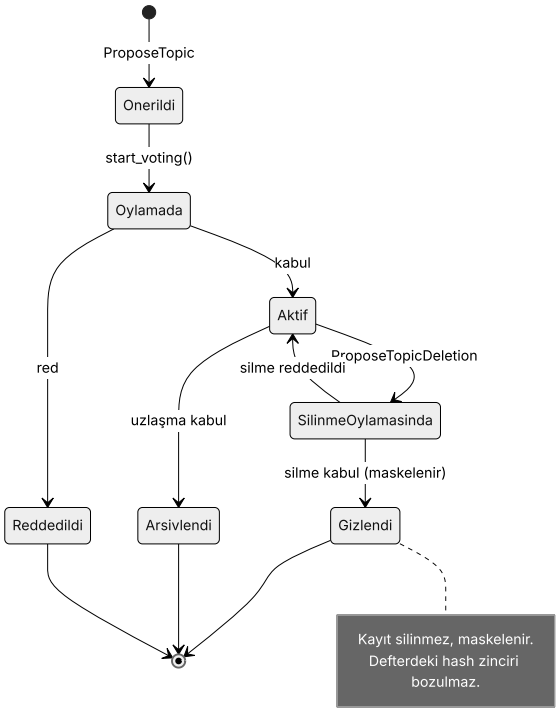
Konunun yaşam döngüsü State deseniyle yönetilir. Her durum (`Proposed`, `InVoting`, `Active`, `InDeletionVote`, `Archived`, `Hidden`, `Rejected`) yalnızca kendisinden geçerli olan geçişleri gerçekler. Temel sınıftaki varsayılan davranış, geçişi açıklayıcı bir mesajla reddetmektir (“Arşivlendi durumundaki bir konu için silme teklifi yapılamaz”). Durumlar ayrıca `can_discuss`, `can_branch`, `can_be_changed` ve `is_visible` bayraklarını taşır. Arayüz “cevap kutusu gösterilsin mi” sorusunu bu bayraklara sorar, durum adını karşılaştırmaz.

Neden State, neden Strategy değil?

Ders notundaki hızlı test: “Nesne kendi kendine davranış değiştiriyor mu?” Konu evet: bir oylama kabul edildiğinde *konunun kendisi* `InVoting` 'den `Active` 'e geçer ve bundan sonra mesaj kabul eder. Bu geçişi dışarıdan kimse seçmez; geçişin kuralı durumun içindedir. Oylama algoritması ise tersine, dışarıdan (yönetmelik tarafından) seçilir ve oylama boyunca değişmez. Bu yüzden konuya State, oylamaya Strategy uygulanmıştır. Durumlar durumsuz (stateless) olduğu için her biri tek bir paylaşılan örnektir; bu, Flyweight'e benzeyen ama ayrı bir desen olarak adlandırılmayı gerektirmeyen bir ayrıntıdır.



Şekil 4. Composite ve State (sınıf diyagramı).

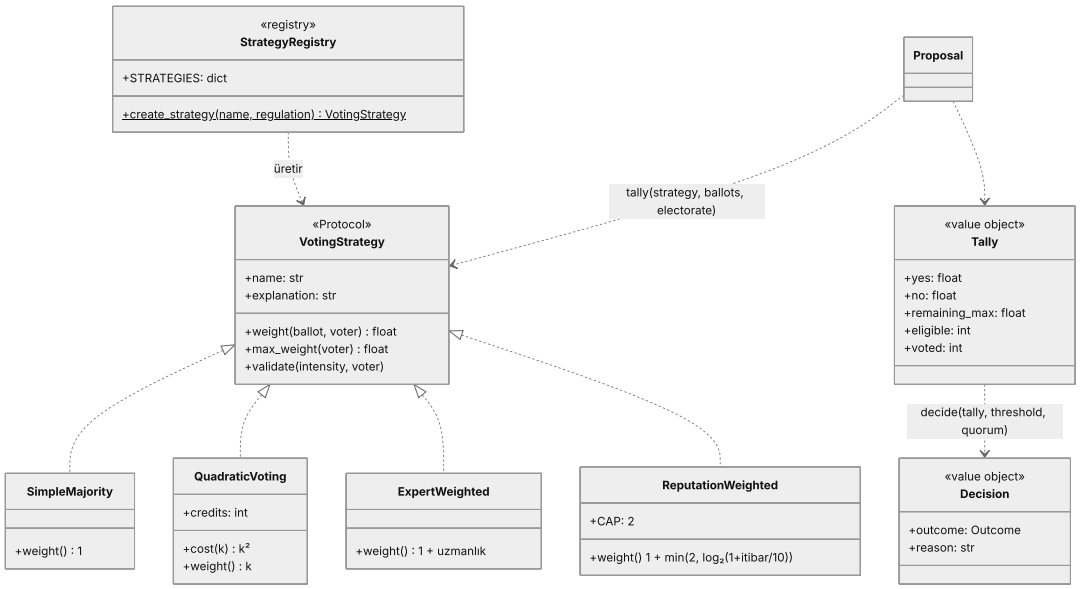


Şekil 5. Konu yaşam döngüsü (durum diyagramı). Gizlenen ya da reddedilen konu silinmez; kaydı ve defterdeki izleri kalır.

6.2 Strategy ve Registry: oylama algoritmaları

Dört oylama algoritması aynı `VotingStrategy` sözleşmesini uygular: bir pusulanın ağırlığı (`weight`), bir seçmenin taşıyabileceği en yüksek ağırlık (`max_weight`, erken karar için gerekir) ve pusula kuralı (`validate`). Sözleşme `abc.ABC` yerine `typing.Protocol` ile yazılmıştır. Gerçeklemeler bir temel sınıftan türemek zorunda değildir, yapısal olarak uymaları yeterlidir (Ramalho, *Fluent Python*, Bölüm 10). Hangi teklif türünün hangi stratejiyle oylanacağı koda gömülü değildir, `Regulation`'da durur ve oylamayla değiştirilebilir. Stratejiler adlarından bir Registry sözlüğüyle üretilir.

STRATEJİ	PUSULA AĞIRLIĞI	TAVAN	VARSAYILAN KULLANIM VE AMAÇ
Basit çoğunluk	1	1	Yeni konu, alt konu. Diğerlerinin karşılaştırıldığı temel çizgi.
Karesel	k oy; bedeli k^2 kredi	3 (9 kredi)	Konu silme, mesaj gizleme, yönetmelik. Yoğun azınlığın sesi (Bölüm 8).
Uzman ağırlıklı	1 + uzmanlık düzeyi	4	Düzenleme, uzmanlık tanıma. Uzmanlık satın alınamaz, oylamayla verilir.
İtibar tabanlı	$1 + \log_2(1 + \text{itibar}/10)$	3	Uzlaşma (konuyu kapatma). Logaritma ve tavan, tek bir “balinanın” sonucu belirlemesini engeller.



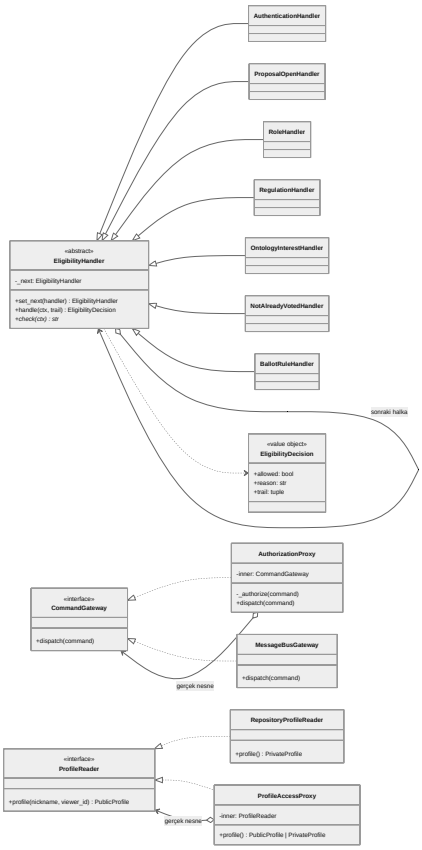
Şekil 6. Strategy: oylama algoritmaları ve Registry (sınıf diyagramı).

6.3 Chain of Responsibility ve Protection Proxy

Bir üyenin bir teklife oy verip veremeyeceği yedi ardışık kurala bağlıdır: **Kimlik** (giriş yapmış mı) → **Oylama açık** → **Rol** (yönetmelik YZ ajanlarına izin veriyor mu) → **Yönetmelik** (yaş sınırı, kendi uzmanlık teklifine oy verememe) → **Ontolojik ilgi** (dondurulmuş seçmen kütüğünde mi) → **Tek oy** → **Pusula kuralı** (stratejinin kuralı, örneğin karesel kredi bütçesi). Her halka bir `EligibilityHandler`'dır. Kural ihlal edilirse zincir durur ve gerekçe döner. Edilmezse halka adını *izi* (`trail`) ekleyip sonrakine devreder. Teklif sayfası bu izi kullanıcıya gösterir: üye hangi denetimlerden geçtiğini ve nerede takıldığını görür.

> **Neden Chain, neden Decorator değil?**
> Ders notundaki hızlı test: "Akış kesilebilir mi?" Burada kesilmesi *gerekir*. Kimliği olmayan birinin yaşını sormanın anlamı yoktur, ilk başarısız halka kararı verir. Decorator ise her zaman içteki nesneye devam eder ve davranış ekler. Ayrıca halkaların sırası anlamlıdır ve tek bir yerde (`build_voting_chain`) okunur. Aynı desen içerik denetiminde ikinci kez kullanılır (boş metin → tehdit → ontoloji notu). Oradaki tek fark, ontoloji halkasının engellememesi, yalnızca not düşmesidir. Bu da ders notundaki "guardrails = Chain of Responsibility" örneğinin karşılığıdır.

Erişim denetimi iki vekille yapılır. `AuthorizationProxy`, web katmanının gördüğü `CommandGateway` arayüzünü gerçekleyen bir Protection Proxy'dir. Her komutu göndermeden önce oturumu ve rolün o komut için yetkisini denetler (örneğin YZ ajanı teklif açamaz), sonra gerçek nesneye (`MessageBusGateway`) devreder. `ProfileAccessProxy` profil okumalarını süzer. Vekil ile gerçek nesne aynı arayüzü uyguladığı için web katmanı hangisiyle konuştuğunu bilmez. Testlerde vekil, gerçek nesne olmadan da sınanabilir.



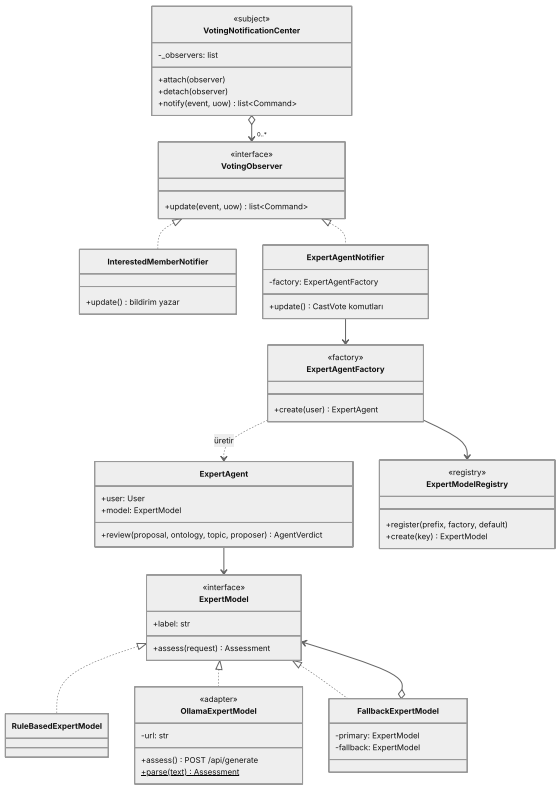
Şekil 7. Chain of Responsibility ve Protection Proxy (sınıf diyagramı).

6.4 Observer, Adapter ve Factory/Registry: YZ uzman ajanları

Bir oylama başladığında iki tür alıcı haberdar olmalıdır: kütükteki üyeler (bildirim) ve kütükteki YZ ajanları (değerlendirip oy vermek için). `VotingNotificationCenter` bir özne (subject) olarak gözlemcileri tutar. `VotingStarted` olayı gelince hepsinin `update` metodunu çağırır. Yeni bir alıcı türü (örneğin e-posta) eklemek, merkezi değiştirmeden yeni bir gözlemci eklemek demektir (açık/kapalı ilkesi).

Gözlemcilerin önemli bir tasarım kararı vardır: `ExpertAgentNotifier` oyu kendisi vermez, bir `CastVote` *komutu döndürür*. Message Bus bu komutu kuyruğun sonuna ekler. Böylece iç içe çağrı (bir işleyicinin içinden yeniden bus'a girmek) olmaz, ajanın oyu insan oyuyla aynı yetki zincirinden geçer ve kendi Unit of Work'ünde işlenir.

Ajanın düşünen kısmı bir `ExpertModel` portudur. Yerel dil modeli sunucusu Ollama'nın HTTP arayüzü bu porta `OllamaExpertModel` ile uyarlanır (Adapter). Kural tabanlı model aynı porta doğrudan uyar. `FallbackExpertModel`, yerel model kapalıysa ya da yanıtı ayrıştırılamazsa kural tabanlı modele düşer; ders notlarındaki *Cascade* deseninin karşılığıdır. Hangi ajanın hangi modeli kullanacağı kullanıcı kaydındaki anahtardan (`"kural"`, `"ollama:llama3.2"`) `ExpertModelRegistry` ile üretilir.

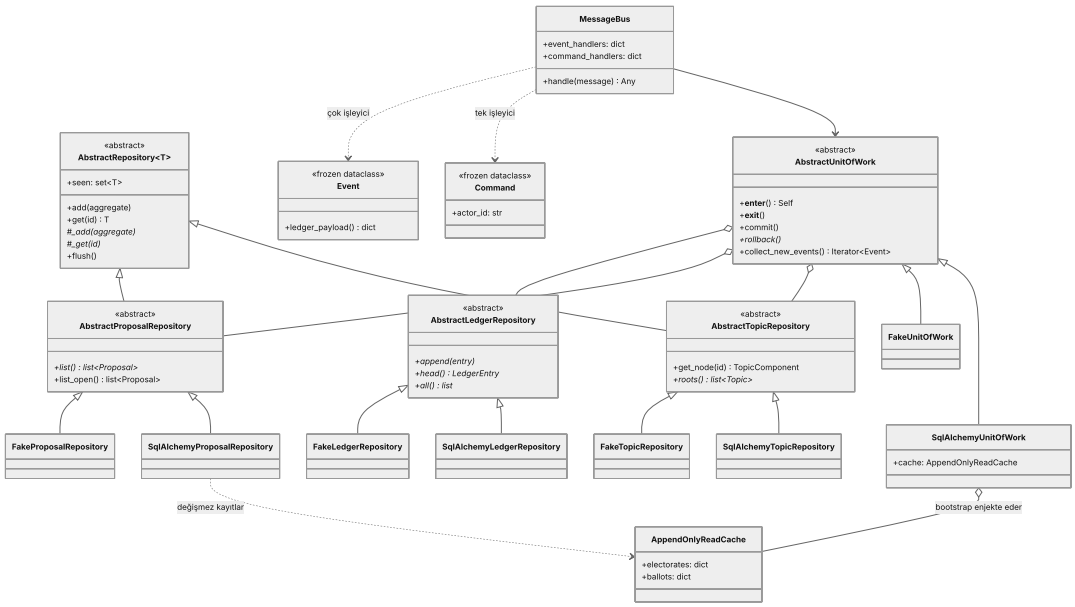


Şekil 8. Observer, Adapter ve Factory/Registry (sınıf diyagramı).

6.5 Repository, Unit of Work ve Message Bus

Her aggregate için soyut bir repository vardır. Bunların SQLAlchemy ve Fake olmak üzere iki gerçeklemesi bulunur. SQLAlchemy tarafı ORM'in sınıf eşlemesini (*classical mapping*) değil, Core ile açık veri eşleyicileri kullanır: alan sınıfları SQLAlchemy'den tamamen habersizdir, satır–nesne dönüşümü repository'de yazılıdır. Repository'ler okunan nesneleri bir kimlik haritasında (identity map) tutar ve yükleme anındaki durumun bir kopyasını saklar. `flush` sırasında yalnızca değişen satırlar yazılır. Hiçbir repository silme metodu sunmaz.

`AbstractUnitOfWork` bir context manager'dır. `with uow:` bloğundan `commit()` çağrılmadan çıkılırsa her şey geri alınır. Commit'ten sonra, repository'lerin gördüğü (`seen`) aggregate'lerde biriken alan olayları `collect_new_events()` ile toplanıp Message Bus'a verilir. Bus, komutların tek, olayların sıfır ya da daha çok işleyicisi olduğu kuralını uygular. Bir olay işleyicisinin hatası günlüğe yazılır ama diğer işleyicileri durdurmaz. Her olay, `record_in_ledger` adlı “her şeyi dinleyen” işleyiciyle deftere de yazılır.



Şekil 9. Repository, Unit of Work ve Message Bus (sınıf diyagramı). `AppendOnlyReadCache`, değişmez kayıtlar (seçmen kütükleri, oylar) için motor başına bir kez üretilip enjekte edilen okuma önbelleğidir.

6.6 Bilinçli olarak kullanılmayanlar

Ders notlarındaki “erken soyutlama: tek gerçeklemesi olan arayüz” uyarısı bir ölçüt olarak alınmıştır. Kurultay'daki her soyutlamanın en az iki gerçeklemesi vardır: repository'lerde SQLAlchemy ve Fake, YZ modellerinde Ollama ve kural, saatte sistem ve elle ilerletilen saat, defter düğümlerinde JSON Lines ve bellek içi, oylamada dört strateji. Aşağıdaki desenler ise değerlendirilip **kullanılmamıştır**:

- **Singleton:** Gizli global durum yaratır (bkz. Bölüm 4.2'deki öz eleştiri). Tek örnek gereken her şey `bootstrap`'ta bir kez üretilip enjekte edilir.
- **Abstract Factory, Builder:** Ürün aileleri ya da çok adımlı kurulum yok. `Proposal.open` gibi adlandırılmış kurucular (Factory Method) yeterli.
- **Decorator:** Yeniden deneme ve günlük gibi kesişen işler için düşünüldü. Bus'ın olay yalıtımı ve Cascade bu ihtiyacı karşıladığı için eklenmedi.
- **Tam olay kaynaklama (event sourcing):** Defter her olayı tutar, ama sistemin durumu defterden yeniden kurulmaz. Defter bir denetim kaydıdır, gerçeğin kaynağı veritabanıdır. Bu, karmaşıklığı ödevin gerektirdiği düzeyde tutan bilinçli bir sınırdır (Bölüm 16).

---

7 Yönetişim: her şey bir teklif

Kurultay'da durumu değiştiren hiçbir komut, değişikliği doğrudan yapmaz. Komut bir `Proposal` açar, değişiklik ancak teklif kabul edilirse uygulanır. Kabul edilen teklifin etkisi, teklif türünden etki fonksiyonuna giden bir Registry sözlüğüyle seçilir. Bu sayede yeni bir teklif türü eklemek mevcut işleyicileri değiştirmeyi gerektirmez. Varsayılan yönetmelikte türler, stratejiler ve eşikler şöyledir. Hepsi oylamayla değiştirilebilir:

TEKLİF TÜRÜ	KABUL EDİLİRSE	STRATEJİ	EŞİK
Yeni konu	Konu *Oylamada* → *Aktif*; red edilirse *Reddedildi* (kaydı kalır)	Basit çoğunluk	1/2

TEKLİF TÜRÜ	KABUL EDİLİRSE	STRATEJİ	EŞİK
Alt konu	Alt konu üst konunun altında açılır (Composite)	Basit çoğunluk	1/2
Düzenleme	Başlık ve açıklamanın yeni sürümü eklenir; eski sürüm kalır	Uzman ağırlıklı	1/2
Konu silme	*Silinme oylamasında* → *Gizlendi* (maske); red edilirse *Aktif*'e döner	Karesel	3/5
Mesaj gizleme	Mesaj maskelenir; metni silinmez, özeti defterde kalır	Karesel	3/5
Uzlaşma	Konu *Arşivlendi*; aynı konudaki rakip teklifler düşer	İtibar tabanlı	1/2
Yönetmelik	Yönetmeliğin yeni sürümü yürürlüğe girer	Karesel	2/3
Uzmanlık	Üyeye bir kavramda uzmanlık düzeyi verilir	Uzman ağırlıklı	1/2

7.1 Seçmen kütüğü: oylama açıldığında dondurulur

Ödev, bir teklifi yalnızca o konunun ontolojisiyle eşleşen kullanıcıların oylamasını ister. İlk taslakta bu denetim oy anında yapılıyordu: oy veren kişinin ilgi alanları teklifin kavram komşuluğuyla kesişiyor mu? Bir değerlendirici şu soruyu sorar: “Bir grup, oylama başladıktan sonra ilgi alanlarına o kavramı ekleyip oylamaya akın ederse ne olur?” Ontolojik kısıt bu durumda bir kapı değil, bir formalite olurdu.

Çözüm, gerçek seçimlerdeki seçmen kütüğüdür. Teklif açıldığı anda `build_electorate` o an ilgili olan üyeleri ve YZ ajanlarını, itibar ve uzmanlık düzeyleriyle birlikte bir `Electorate` değer nesnesine yazar. `Proposal.freeze_electorate` bunu yalnızca bir kez kabul eder; ikinci çağrı `InvalidTransition` fırlatır. Kütük, append-only `electorate_voters` tablosunda saklanır. Yetki zincirinin “Ontolojik ilgi” halkası artık kütüğe bakar. Sonradan ilgilenen üyeye neden oy veremediği açıkça söylenir: “Seçmen kütüğü oylama açıldığında belirlendi…”. Dondurmanın ikinci bir yararı vardır: paydadaki seçmen sayısı ve olası en yüksek ağırlık oylama boyunca sabit kalır, dolayısıyla yüzde ve eşik hesapları kararlıdır.

7.2 Erken karar ve yeter sayı

Bir oylamanın son tarihini beklemek, sonucu zaten belli olan teklifleri günlerce askıda bırakır. Kurultay, sonuç *matematiksel olarak kesinleştiği* anda oylamayı kapatır. E evet ağırlığı, H hayır ağırlığı, K henüz oy vermemiş seçmenlerin taşıyabileceği en yüksek toplam ağırlık ve θ eşik olsun:

Erken kabul: $E \,/\, (E + H + K) \geq \theta$ **Erken red:** $(E + K) \,/\, (E + H + K) < \theta$

Kalan seçmenlerin bundan sonra vereceği oylar evet tarafına a, hayır tarafına b ağırlık eklesin ($a + b \leq K$). Son evet payı $(E + a) \,/\, (E + H + a + b)$, $a = 0$, $b = K$ iken en küçük, $a = K$, $b = 0$ iken en büyük değerini alır. Bu iki uç değer yukarıdaki iki koşuldur. Dolayısıyla erken verilen bir karar, kalan oylar nasıl dağılırsa dağılsın değişmez.

Karşılaştırmalar kayan nokta yerine `fractions.Fraction` ile yapılır. 0,6 ile 3/5'in kayan noktadaki farkı bir oylamanın sonucunu değiştirmemelidir. Son tarih geldiğinde ise önce yeter sayıya (kütüğün en az %30'u oy vermiş olmalı) bakılır, sonra kullanılan oylarda eşiğe. Yeter sayı yalnızca son tarihte uygulanır, çünkü erken kabul zaten kütüğün *tamamının* olası ağırlığına göre bir çoğunluk gerektirir.

7.3 Kuralları da topluluk değiştirir

`Regulation` dondurulmuş bir değer nesnesidir. `with_change` mevcut sürümü değiştirmez, sürüm numarası bir artmış yeni bir nesne döndürür. Eski sürümler append-only bir tabloda kalır. Oylamayla değiştirilebilen kuralların her birinin alt ve üst sınırı vardır (`ChangeableRule`): yeter sayı %0'a indirilemez, YZ ajanı ağırlığı %100'ü geçemez. Demo verisinde topluluk yeter sayıyı %30'dan %15'e indirmeyi kabul etmiş, mesaj gizlemeyi basit çoğunluğa çevirme önerisini ise reddetmiştir. YZ ajanı ağırlığını %75'e çıkarma teklifi demo anında hâlâ oylamadadır.

Nerede?
`domain/proposal.py` (`open`, `freeze_electorate`, `cast`, `try_close`) · `domain/voting.py` (`tally`, `decide`) · `domain/eligibility.py` · `domain/regulation.py` · testler: `test_frozen_electorate…`, `test_early_acceptance_when_remaining_votes…`, `test_deadline_closes_with_quorum_rule`, `test_regulation_change_creates_new_version`

---

8 Azınlığın korunması

"Çoğunluğun azınlığı ezmesini engellemek" ödevin açık isteğidir. Kurultay bunu tek bir mekanizmaya bırakmaz. Karesel oylama yoğunluğu ölçer, nitelikli eşikler (3/5, 2/3) geri dönüşü zor kararlarda geniş uzlaşma ister, ontolojik kütük ise konuyla ilgisi olmayan kalabalığın oylamaya taşınmasını önler.

Öz eleştiri: kıtlığı olmayan karesel oylama, karesel oylama değildir
İlk taslakta her seçmene *her teklif için ayrı* 9 kredi veriliyordu. Bu durumda rasyonel bir seçmen için baskın strateji her oylamada 3 oy kullanmaktır, çünkü kredi başka hiçbir yerde işe yaramaz. Herkes 3 oy kullanınca bütün ağırlıklar üçle çarpılır ve yöntem basit çoğunluğa indirgenir. Azınlığı koruyan şey yöntemin kendisi değil, çoğunluğun üşengeçliği olur. Lalley ve Weyl'in (2018) karesel oylamasını anlamlı kılan şey, kredinin *kıt* ve kararlar arasında *paylaşılan* olmasıdır: bir konuda sesini yükselten, başka bir konuda kısmak zorundadır.

Düzeltme: İlk oy herkese bedavadır, böylece kimsenin sesi kredi yüzünden kısılmaz. 2 ya da 3 oy kullanmanın ek bedeli ($k^2 - 1 = 3$ ya da 8 kredi), seçmenin 8 kredilik bütçesinden düşer. Bu bütçe o an açık olan **bütün karesel oylamalar arasında paylaşılır**, oylama kapandığında harcanan kredi geri gelir. Kural alan katmanında (`QuadraticVoting.validate`) durur. Harcanan kredi servis katmanında tek bir SQL toplamıyla hesaplanıp yetki zincirinin "Pusula kuralı" halkasına verilir. İki yeni birim testi ve bir entegrasyon testi, SQL toplamının soyut tanımla aynı sonucu verdiğini de denetler.

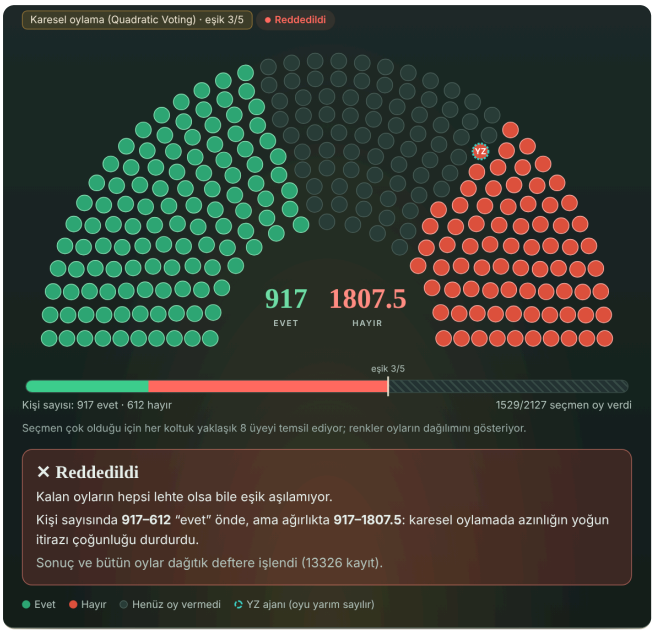
```
# domain/voting.py — QuadraticVoting (credits = 9, budget = credits − 1 = 8)
def extra_cost(self, intensity: int) -> int:
    return self.cost(intensity) - 1                    # ilk oy bedava: 0, 3, 8

def validate(self, intensity: int, voter: VoterProfile, spent: int = 0) -> None:
    if intensity < 1 or self.cost(intensity) > self.credits:
        raise InvalidValue("Karesel oylamada en fazla 3 oy (9 kredi) kullanılır.")
    left = max(0, self.budget - spent)                 # spent: açık oylamalardaki Σ(k² − 1)
    if self.extra_cost(intensity) > left:
        raise InvalidValue("Ek kredin yetmiyor ... 1 oy her zaman kullanılabilir.")
```

8.1 Canlı örnek: kişi sayısında kaybeden azınlık

Uygulamanın "Meclis" sayfası bir oylamayı gerçek komutlarla, koltuk koltuk oynatır. Aşağıdaki senaryoda sert bir mesajın gizlenmesi oylanmaktadır (karesel, eşik 3/5, 2 127 kişilik dondurulmuş

kütük). Çoğunluk konuyu az önemsediği için kredisini harcamadan 1'er oy verir. Mesajın kalmasını isteyen azınlık 3'er oy kullanmak ister.



Şekil 10. Canlı oylamanın sonucu. Kişi sayısında 917–612 "evet" önde, ağırlıkta 917–1 807,5 "hayır" önde. Erken red: kalan oyların hepsi lehte olsa bile eşik aşılamıyor.

	EVET	HAYIR	EVET PAYI
Kişi sayısı	917	612	%59,97
Karesel ağırlık	917	1 807,5	%33,7

Basit çoğunlukla ve 1/2 eşikle mesaj gizlenirdi. Karesel oylamada azınlığın yoğun itirazı çoğunluğu durdurur. İki ayrıntı önemlidir:

- Hayır ağırlığı 3 × 612 = 1 836'nın altında kalmıştır. Başka açık karesel oylamalarda kredi harcamış bazı üyeler bu oylamaya daha az oyla katılabilmiştir. Bu, yeni kıtlık kuralının canlı veride işlediğinin kanıtıdır.
- Kütükteki YZ ajanı (şekilde "YZ" koltuğu) "mesaj sert olabilir ama tehdit değil" gerekçesiyle hayır demiş ve oyu yönetmelikteki oranla 0,5 sayılmıştır.

Çoğunluk da 3 oy kullansaydı sonuç değişirdi; ama o zaman 8 kredilik bütçesinin tamamını bu tek oylamaya harcamış olurdu. Karesel oylamanın vaat ettiği şey budur: yoğunluk bedavaya gelmez.

8.2 Diğer korumalar

- **Nitelikli eşikler:** Geri dönüşü zor kararlar (gizleme, silme: 3/5; kural değişikliği: 2/3) salt çoğunlukla alınamaz.
- **Ağırlık tavanları:** Uzman oyu en fazla 4, itibar oyu en fazla 3 sayılır, itibar logaritmiktir. Hiçbir tek üye bir oylamayı tek başına belirleyemez.
- **Tek içerik yasağı tehdittir:** Bir görüşün "karşı görüş" olması gizleme sebebi değildir. Kural tabanlı YZ ajanı tehdit ya da reklam olmayan bir mesajın gizlenmesine her zaman hayır der.
- **Gizleme silme değildir:** Yanlışlıkla gizlenen bir içerik bile defterde kalır. Maske ve onu koyan teklif herkes tarafından görülebilir.

---

9 Değiştirilemez dağıtık defter

Sistemdeki her alan olayı (kayıt, teklif, oy, mesaj, gizleme, kapanış) bir `LedgerEntry` olarak deftere eklenir. Her kayıt bir öncekinin özetini taşır:

$$\text{hash}_i = \text{SHA-256}(\ i \mid \text{zaman} \mid \text{tür} \mid \text{kanonik JSON veri} \mid \text{hash}_{i-1}\ ), \qquad \text{hash}_{-1} = 0^{64}$$

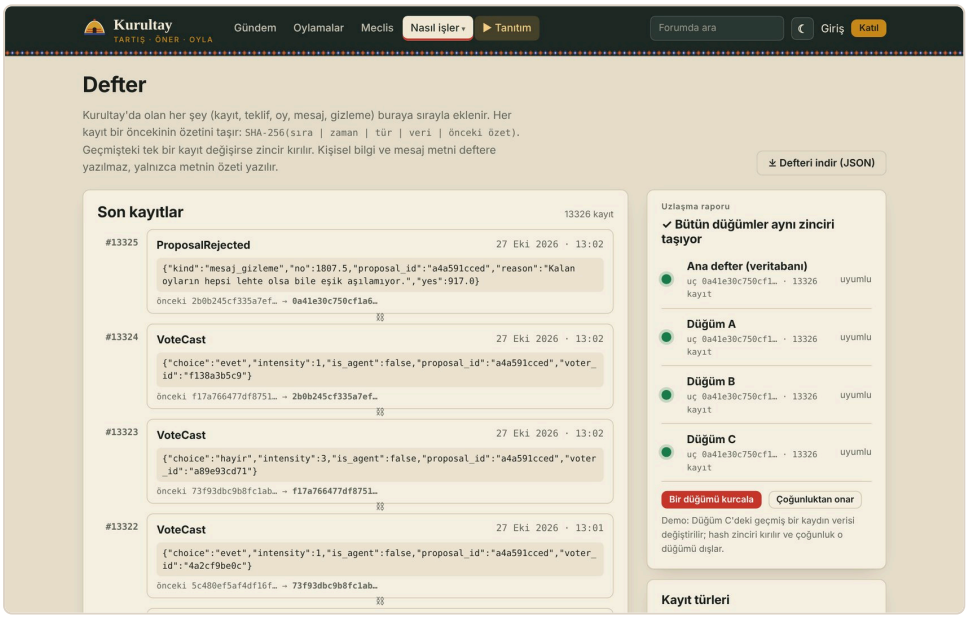
Veri, anahtarları sıralı kanonik JSON'a çevrilir; böylece aynı olay her makinede aynı özeti üretir. Geçmişteki tek bir kaydın tek bir karakteri değişirse o kaydın özeti tutmaz ve ondan sonraki bütün halkalar kopar. `verify_chain` ilk bozuk kaydın sırasını raporlar. Kişisel veri ve mesaj metinleri deftere yazılmaz. Mesajın yerine metninin SHA-256 özeti yazılır, böylece gizlenen bir mesajın sonradan değiştirilmediği kanıtlanabilir ama içeriği defterden okunamaz.

9.1 Üç katmanlı append-only güvencesi

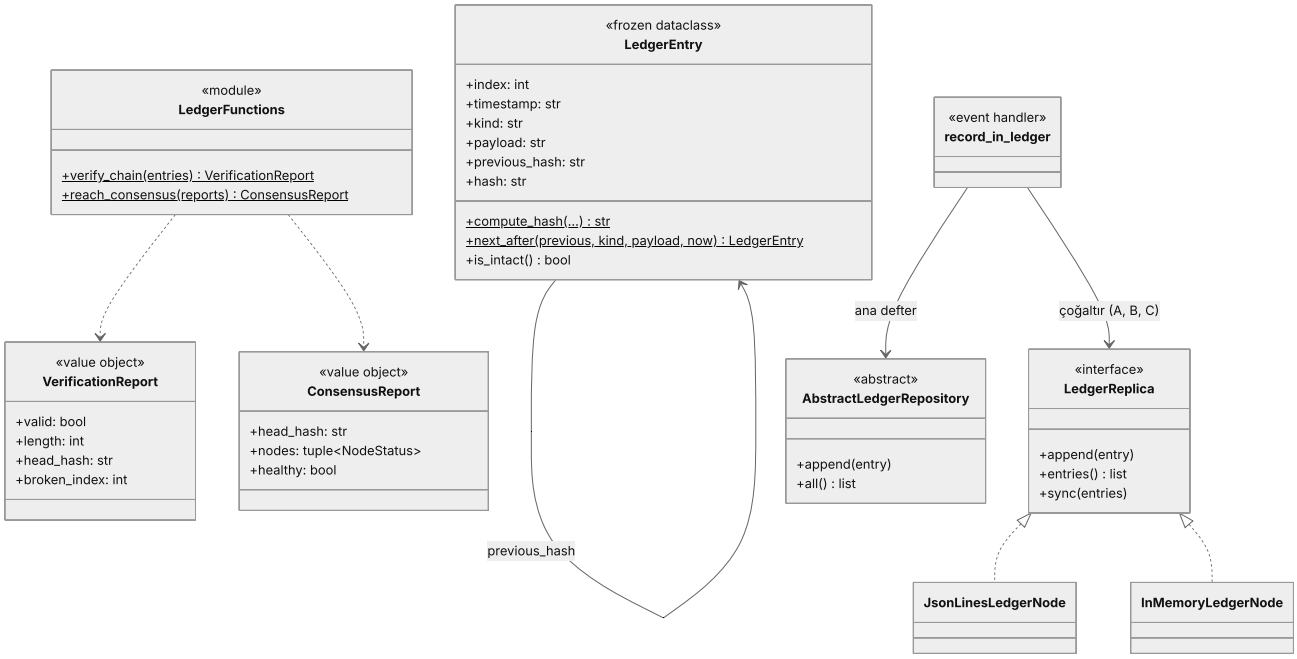
1. **Alan:** `ledger.py`'de güncelleme ya da silme işlemi yoktur. Gizleme bir `Mask` eklemektir, düzenleme bir `TopicRevision` eklemektir.
2. **Repository:** Hiçbir repository'de silme metodu yoktur ve bu bir mimari testle denetlenir.
3. **Veritabanı:** Yedi tabloda (defter, mesajlar, maskeler, oylar, seçmen kütükleri, konu sürümleri, yönetmelik geçmişi) `BEFORE UPDATE` ve `BEFORE DELETE` tetikleyicileri işlemi `RAISE(ABORT)` ile reddeder. Konular, teklifler, kullanıcılar ve kavramlar silinemez, konunun yazarı ve oluşturulma zamanı değiştirilemez. Uygulama kodunu atlayıp doğrudan SQL yazan biri de geçmişi değiştiremez (entegrasyon testi: `test_database_triggers_forbid_rewriting_history`, 9 farklı SQL ifadesiyle).

9.2 Çoğaltma ve uzlaşma

Her kayıt, veritabanındaki ana deftere ek olarak üç düğüme (A, B, C; ayrı JSON Lines dosyaları) çoğaltılır. `reach_consensus`, geçerli zincirlerin uç özetleri arasında salt çoğunluk arar. Çoğunluktan ayrılan düğüm “kurcalanmış” olarak işaretlenir ve çoğunluğun zinciriyle onarılabilir. Defter sayfasında bir düğümü bilerek kurcalayıp bu tespiti canlı izlemek mümkündür. 13 600 kayıtlık zincirin dört kopyasının doğrulanması 0,7 saniye sürer. Sonuç, defter ya da bir düğüm değişene kadar yeniden kullanılır.



Şekil 11. Defter sayfası: son kayıtlar, her kaydın önceki ve kendi özeti, düğümlerin uzlaşma raporu ve kurcalama gösterimi.



Şekil 12. Dağıtık defter: hash zinciri, düğüm kopyaları ve çoğunluk uzlaşması (sınıf diyagramı).

Not: Düğümler aynı makinede durur ve uzlaşma basit çoğunluktur. Bu, Bizans hatalarına dayanıklı bir dağıtık sistem değil, hash zincirinin ve çoğaltmanın öğretim amaçlı bir modelidir (Bölüm 16).

---

10 YZ uzman ajanları

Ödev, dış YZ modellerinin sisteme "bilgili kullanıcı" olarak katılmasını ister. Kurultay'da bir YZ ajanı ayrı bir alt sistem değil, `yz_ajan` rolüyle kayıtlı bir **kullanıcıdır**. İlgi alanları vardır, seçmen kütüklerine insanlarla aynı kuralla girer, oylama başladığında Observer ile haberdar edilir ve oyunu insanlarla aynı yetki zincirinden geçirir. Ayrı bir yol açılmadığı için ajanlar sistemin hiçbir kuralını atlayamaz.

10.1 Korkuluklar

- **Teklif açamaz:** `AuthorizationProxy`'deki yetki tablosu ajanlara yalnızca oy ve mesaj izni verir. Ajanlar arayüzden kaydedilemez.
- **Oyu tam sayılmaz:** Ajan oyunun insan oyuna oranı yönetmeliktedir (varsayılan %50). Kural tabanlı model bu oranın %50'nin üzerine çıkarılmasına kendisi de karşı oy verir.
- **Kapatılabilir:** Yönetmelikteki `agents_may_vote` kuralı oylamayla kapatılırsa yetki zincirinin "Rol" halkası ajan oylarını reddeder.
- **Gerekçe zorunludur:** Her ajan oyu, modelin tek cümlelik gerekçesi ve hangi modelin kullanıldığı bilgisiyle saklanır ve teklif sayfasında gösterilir.

10.2 Modeller: kural, yerel BDM ve Cascade

Ajan, teklif türünü, başlığı, kavramları, metni ve Kurultay'ın ilkelerini içeren bir istem (prompt) hazırlar. Modelden ilk satırda `KARAR: EVET|HAYIR`, ikinci satırda `GEREKÇE: …` ister. `OllamaExpertModel` bu istemi yerel Ollama sunucusuna gönderir ve yanıtı düzenli ifadeyle ayrıştırır. Sunucu kapalıysa, zaman aşımı olursa ya da yanıtta KARAR satırı yoksa `ModelUnavailable` fırlatılır ve `FallbackExpertModel` kural tabanlı

modele düşer. Kullanıcı hangi modelin karar verdiğini gerekçenin sonunda görür ("kural tabanlı model; yerel model kapalıydı").

Kural tabanlı model, ilkeleri açıkça kodlayan küçük bir karar listesidir. Tehdit içeren her teklife hayır der. Mesaj gizlemeye yalnızca metin reklamsa (en az iki reklam işareti) evet der, görüş bildiren sert mesajlara hayır der. Yeter sayının %20'nin altına ya da eşiklerin salt çoğunluğa indirilmesine karşı çıkar. Uzmanlık tanıma için önerenin itibarına bakar. Bu model aynı zamanda BDM'nin yenmesi gereken temel çizgidir. Demo verisindeki üç ajandan ikisi kural tabanlı modeli, biri `ollama:llama3.2` anahtarını kullanır. Ollama kurulu değilse bu ajan da Cascade ile kurala düşer, dolayısıyla uygulama hiçbir koşulda ücretli bir servise ihtiyaç duymaz.

Neden bulut BDM API'si yok?

Ödevin bir kısıtı ücretsiz olmasıydı. Daha önemlisi, bir topluluğun oylamasına katılan bir modelin davranışı tekrarlanabilir ve denetlenebilir olmalıdır. Yerel model ve kural tabanlı yedek bu iki koşulu da sağlar. Yeni bir model eklemek (örneğin başka bir yerel sunucu) yalnızca yeni bir Adapter yazıp Registry'ye bir satır eklemektir. Ajan, servis katmanı ve testler değişmez.

Nerede?

`service_layer/agents.py` (`ExpertAgent` , `ExpertModelRegistry` , `ExpertAgentFactory`) · `adapters/llm/ollama.py` , `rule_based.py` · `service_layer/notifications.py` · testler: `test_rule_based_model_protects_speech_and_minorities` , `test_ollama_response_parsing` , `test_fallback_model_cascades_when_local_llm_is_down` , `test_agents_stay_silent_when_regulation_forbids`

---

11 Ölçüm ve değerlendirme

Ders notlarının kuralı açıktır: "baseline olmadan sayı anlamsız". Bir yöntemin iyi olup olmadığı, aynı veride en basitten başlayan bir temel çizgi merdiveniyle karşılaştırılarak anlaşılır: rastgele → çoğunluk sınıfı → naif kural → bizim yöntem. Ölçüm, depodaki `python -m kurultay.evaluation` komutuyla yeniden üretilir. Sonuçlar `docs/olcum.json`'a yazılır ve uygulamanın Mimari sayfasında da gösterilir. Rastgele temel çizgi sabit tohumlu bir üreteç kullanır, dolayısıyla tablo her çalıştırmada aynıdır.

11.1 Tehdit kuralı: temel çizgi merdiveni

Etiketli set ilk sürümden *aynen* taşınmıştır ve kural bu sete göre ayarlanmamıştır. Sette kuralın kaçıracağı bilinen örtük tehditler ve yanlış alarm vereceği bilinen mecazlar bilerek bulunur. Set 60 cümledir (20 tehdit, 40 tehdit değil). Maliyet sütunu Bölüm 3.2'deki fonksiyondur: 2 × YP + YN.

YÖNTEM	AÇIKLAMA	DP	YP	YN	DN	KESİNLİK	DUYARLILIK	F1	F0.5	MALİYET
Rastgele (oranla)	Sınıf oranına göre yazı tura	6	16	14	24	0,27	0,30	0,29	0,28	46
Çoğunluk sınıfı	Her şeye "tehdit değil"	0	0	20	40	—	0,00	0,00	0,00	20
Naif anahtar kelime	11 şiddet kökünden biri geçiyorsa	13	19	7	21	0,41	0,65	0,50	0,44	45

YÖNTEM	AÇIKLAMA	DP	YP	YN	DN	KESİNLİK	DUYARLILIK	F1	F0.5	MALİYET
Bizim kural	**Açık kalıp ya da 1. tekil şiddet fiili + hedef**	**14**	**2**	**6**	**38**	**0,88**	**0,70**	**0,78**	**0,83**	**10**

	TAHMİN: TEHDİT	TAHMİN: DEĞİL
Gerçek: tehdit	14 (DP)	6 (YN)
Gerçek: değil	2 (YP)	38 (DN)

Bizim kuralın karışıklık matrisi (n = 60).

Yorum. Kural, F0.5'te naif yöntemin neredeyse iki katına çıkar ve kesinlik 0,88'dir. Tablonun daha öğretici satırı naif yöntemdir. Duyarlılığı bizim kurala yakın (0,65) olduğu için ilk bakışta "işe yarıyor" sanılabilir, ama 19 masum cümleyi engeller. Maliyeti (45), hiçbir şeyi engellemeyen çoğunluk sınıfının maliyetinden (20) bile kötüdür. Ders notlarındaki "doğruluk tuzağı"nın bir benzeri burada duyarlılıkta görülür: tek bir metriğe bakmak, sansür üreten bir sistemi başarılı gösterebilir.

Hata analizi: kural nerede yanılıyor?

TÜR	CÜMLE	NEDEN KAÇTI / NEDEN YANLIŞ ALARM?
YN	Ailenizi tek tek bulup gebertiriz.	Açık kalıp listesi yalnızca 1. tekil şahsı ("gebertirim") içeriyor; 1. çoğul eksik.
YN	Kafana sıkacağım.	Hedef, zamir değil iyelik ekiyle ("kafana") veriliyor; hedef listesinde "kafanı" var, "kafana" yok.
YN	Arabana bomba koyacağım.	Şiddet fiil kökü yok; şiddet nesnede ("bomba"). Kural fiile bakıyor.
YN	Senin gibileri asarım ben, sıran gelecek.	Hedef zamiri yok ("senin gibileri"); "sıran gelecek" örtük tehdit.
YN	Yarın okulun önünde bekliyor olacağım, çocuğunu bir daha göremezsin.	Tamamen örtük: şiddet fiili yok. Bağlam gerektirir; kuralın doğal sınırı.
YN	Mahallede seni gören dövecek, ben söyledim.	Şiddet üçüncü kişilere yüklenmiş; 1. tekil fiil yok.
YP	Seni tavlada döverim, gel bir el atalım.	Mecaz; "döverim" + "seni" kalıbına uyuyor. Bağlam (tavla) görülmüyor.
YP	"Seni öldürürüm" diye yazan hesabı kınıyorum.	Alıntı içindeki tehdit; tırnak içi ayrıştırılmıyor.

İlk iki hata kalıp listesine birer ek yaparak düzeltilebilir. Ancak bu, kuralı test setine göre ayarlamak (aşırı uyum) olurdu ve 0,83'ü anlamsız kılardı. Doğru yol, önce bağımsız ikinci bir etiketleyiciyle seti büyütmek ve etiketleyiciler arası uyumu (Cohen κ) ölçmek, sonra kuralı ayrı bir geliştirme setinde iyileştirip test setinde bir kez ölçmektir. Son dört hata ise kural yaklaşımının yapısal sınırıdır. İleride bir modelin kuralı yenebileceği yer burasıdır, ama o model de bu tablodaki 0,83'ü açık farkla geçmek zorundadır.

11.2 Kavram önerisi: konu → ana bölüm

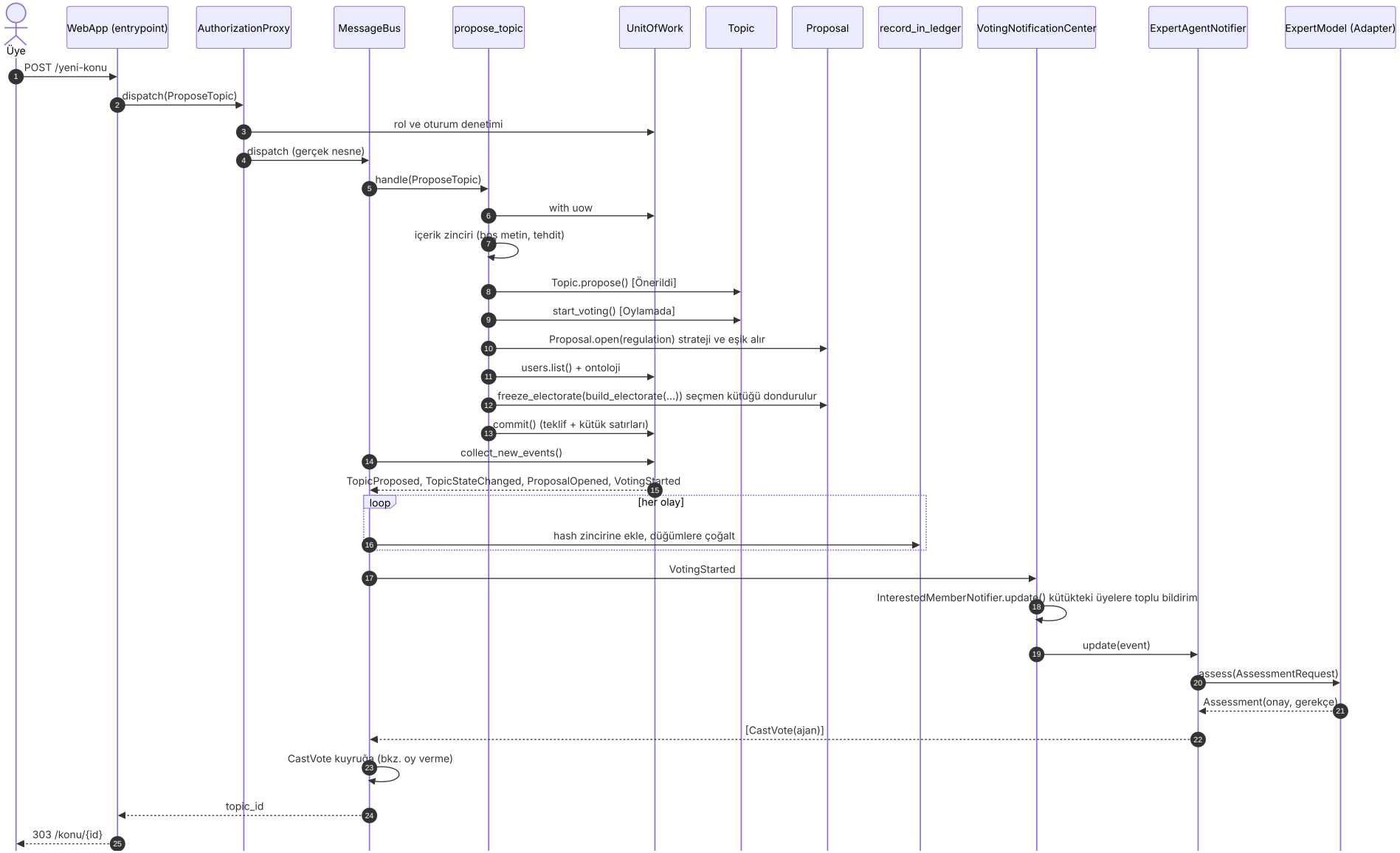
İkinci ölçüm, `Ontology.suggest`'in bir konuyu doğru ana bölüme (ontoloji ağacının kökü) yerleştirip yerleştiremediğidir. Veri, demo verisindeki 25 konunun başlık ve metinleridir, doğru etiket konuyu açanın seçtiği kavramın köküdür (11 sınıf). Çok sınıflı ve dengesiz olduğu için ana metrik makro F1'dir; küçük bölümler de eşit söz sahibidir.

YÖNTEM	AÇIKLAMA	DOĞRULUK	MAKRO F1
Rastgele (oranla)	Bölüm oranlarına göre rastgele	0,12	0,12
Çoğunluk sınıfı	Her konuya en sık bölüm (“Ekonomi”)	0,20	0,03
Bizim kural	**Anahtar kelime örtüşmesi**	**0,68**	**0,60**

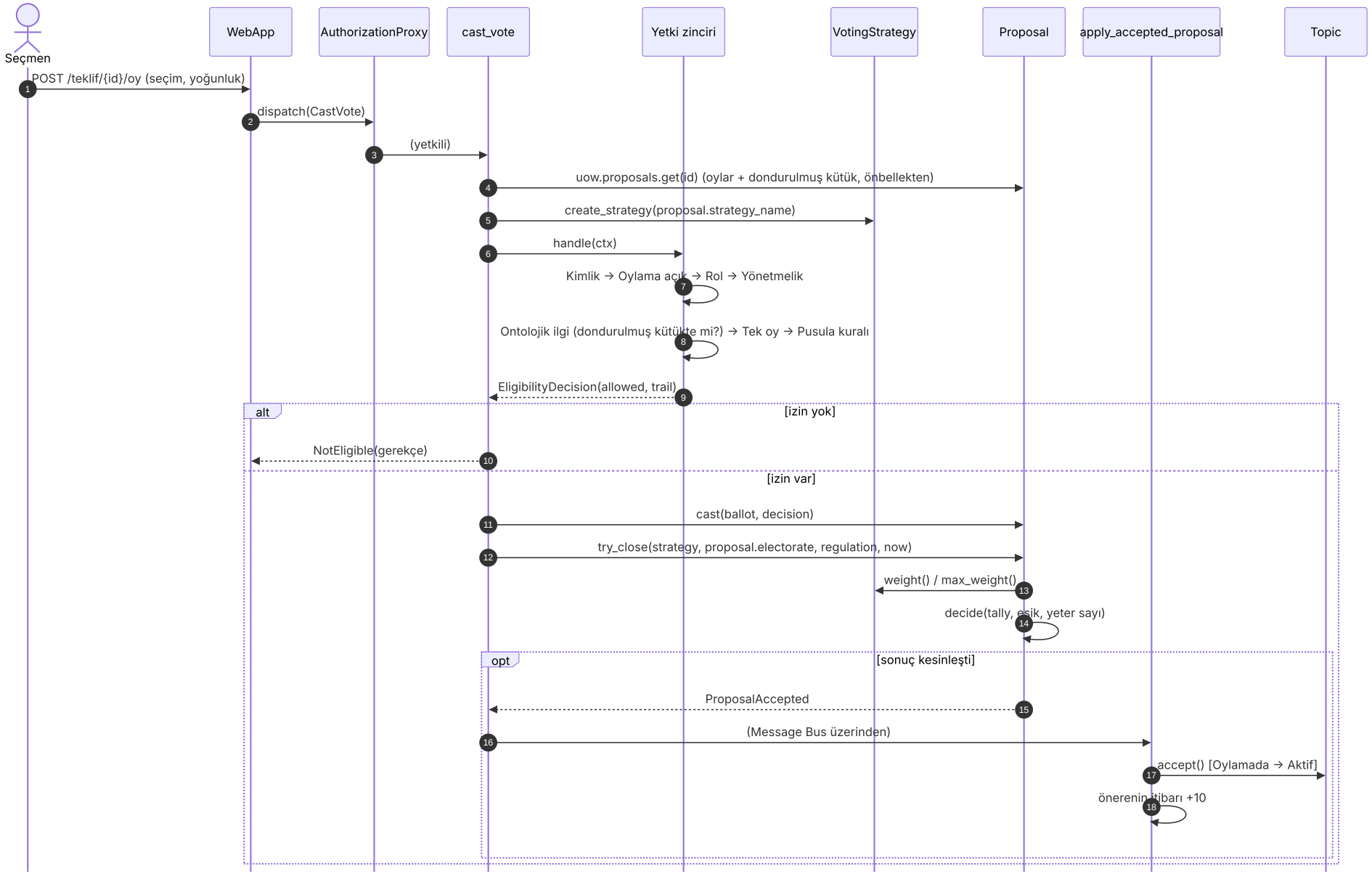
Geçerlilik uyarısı. Demo konuları ve ontoloji anahtar kelimeleri aynı kişi tarafından yazıldığı için bu sayı iyimserdir ve bir genelleme iddiası taşımaz. Bileşen karar vermediği, yalnızca öneri notu düştüğü için bu düzeydeki bir belirsizlik kabul edilebilir (Bölüm 3).

12 Davranış: sıralama diyagramları

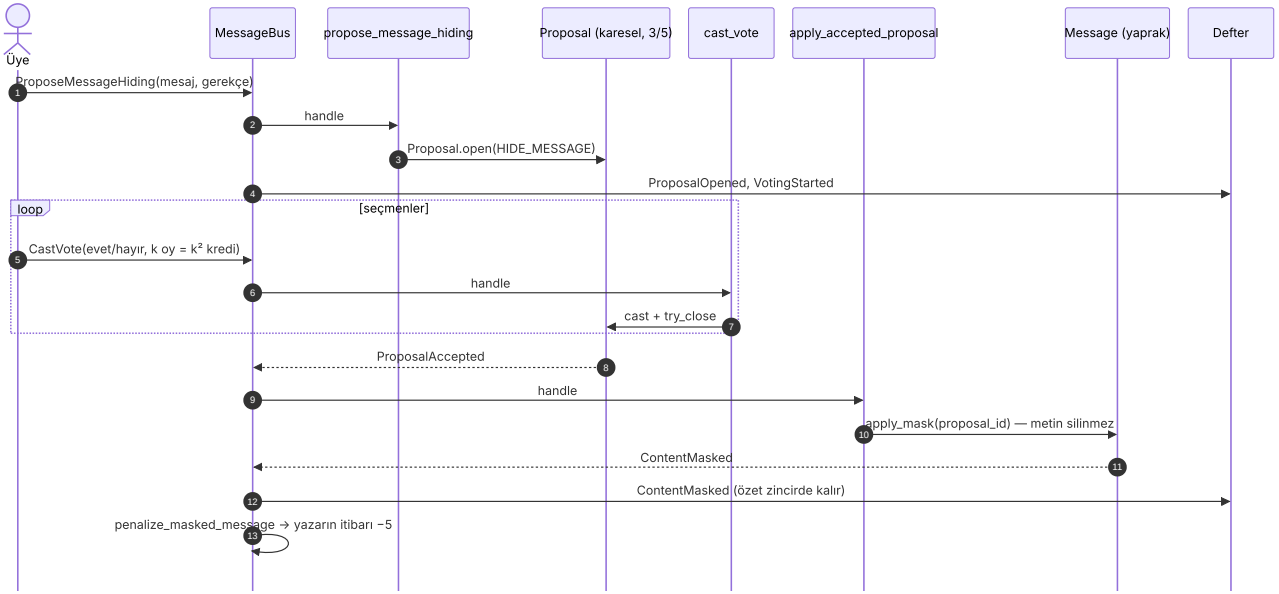
Sınıf diyagramları yapıyı, sıralama diyagramları ise bir isteğin katmanlar arasındaki yolculuğunu gösterir. Diyagramlar koddaki gerçek sınıf ve fonksiyon adlarını kullanır. Ortak örüntü şudur: HTTP isteği bir komuta dönüşür ve önce Protection Proxy'den, sonra Message Bus'tan geçer. İşleyici kendi Unit of Work'ü içinde aggregate'leri değiştirir. Commit'ten sonra toplanan olaylar deftere yazılır ve gözlemcilere dağıtılır. Beş kullanım durumundan profil görüntüleme, kişisel veriyle birlikte Bölüm 5'te (Şekil 3) verilmiştir. Diğer dördü izleyen yatay sayfalardadır: konu önerisi (Şekil 13), oy verme ve sonuçlanma (Şekil 14), karesel oylamayla mesaj gizleme (Şekil 15) ve defter kaydıyla doğrulama (Şekil 16).



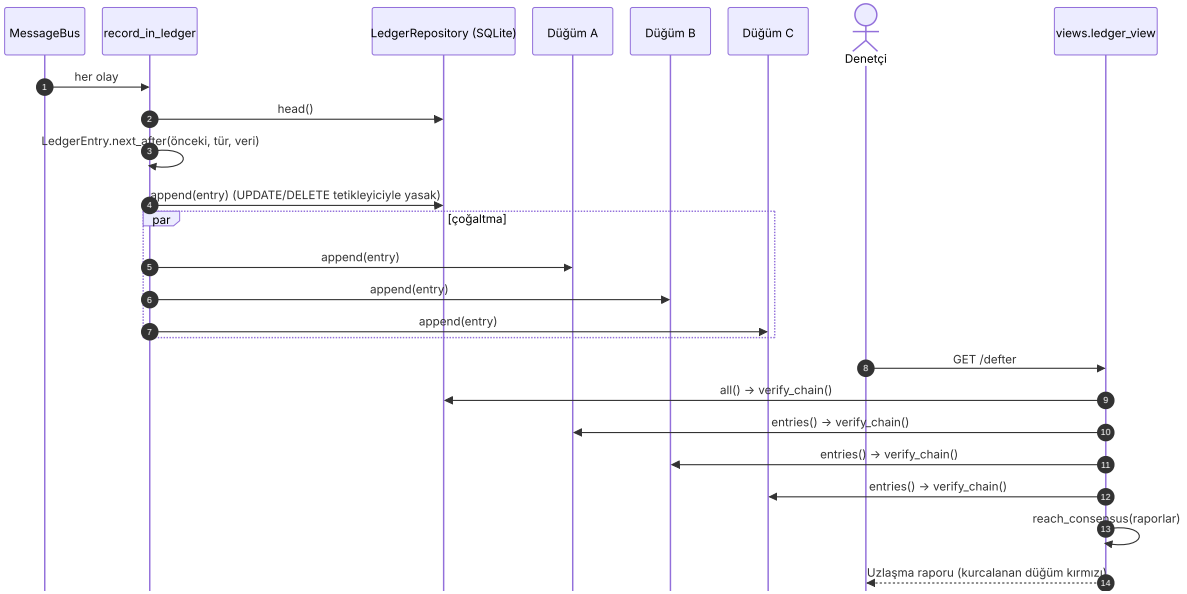
Şekil 13. Konu önerisi (sıralama diyagramı). YZ ajanının oyu bir işleyicinin içinden doğrudan verilmez; gözlemci bir `CastVote` komutu döndürür ve Bus onu kuyruğun sonunda, kendi Unit of Work'ünde işler.



Şekil 14. Oy verme ve sonuçlanma (sıralama diyagramı). İzin yoksa zincir gerekçeyle durur. İzin varsa oy eklenir ve sonuç kesinleştiyse teklif kendini kapatır.



Şekil 15. Mesaj gizleme (sıralama diyagramı). Mesaj silinmez, bir `Mask` eklenir; deftere metnin kendisi değil özeti yazılır.



Şekil 16. Defter kaydı, çoğaltma ve doğrulama (sıralama diyagramı).

13 Test stratejisi

Testler Cosmic Python'un "yüksek vites / düşük vites" önerisine göre dağıtılmıştır. Davranışın büyük kısmı servis katmanında, komut göndererek ve sonucu Fake repository'lerden okuyarak test edilir. Böylece testler iç yapıya değil davranışa bağlı kalır ve iç yapı değiştiğinde kırılmaz. Alan testleri yalnızca kenar durumlar (geçersiz geçişler, sayım matematiği) içindir. Gerçek SQLite yalnızca entegrasyon testlerinde, gerçek HTTP yalnızca uçtan uca testlerde kullanılır.

DÜZEY	TEST	NE KULLANIR	NEYİ KANITLAR
Birim (domain + servis + mimari + ölçüm)	118	`FakeUnitOfWork`, 7 Fake repository, `FakeExpertModel`, `ManualClock`, `FakePasswordHasher`	İş kuralları, desenler, mimari kurallar, metriklerin el hesabıyla uyumu
Entegrasyon	20	Bellek içi SQLite, geçici dosyalar	Repository gidiş–dönüşü, tetikleyiciler, defter düğümleri, önbelleğin yeniden yüklemede doğruluğu, SQL toplamı = soyut tanım
Uçtan uca	11	FastAPI `TestClient`, geçici veri klasörü; canlı gösterim için 700 üyelik demo	Sayfalar, giriş, oy verme, PII'nin HTML'de görünmemesi, canlı gösterim
Toplam	**149**	**0 başarısız · ≈32 saniye · `mypy src` : 52 modülde 0 hata**	

Aşağıdaki test, Bölüm 8'deki kıtlık kuralını servis katmanından, yalnızca komut göndererek sınar. Veritabanı yoktur, saat elle ilerletilir:

```
def test_intensity_budget_is_shared_across_open_quadratic_votes_and_freed_on_close(harness):
    members = community(harness, 6)
    topic_id = accepted_topic(harness, members)
    first = harness.run(c.PostMessage(members[1], topic_id, "Sert mesaj 1"))
    second = harness.run(c.PostMessage(members[1], topic_id, "Sert mesaj 2"))
    p1 = harness.run(c.ProposeMessageHiding(members[2], first, "sert"))
    p2 = harness.run(c.ProposeMessageHiding(members[2], second, "sert"))
    harness.run(c.CastVote(members[0], p1, "hayir", 3))          # 8 ek kredinin hepsi
    with pytest.raises(NotEligible, match="Ek kredin yetmiyor"):
        harness.run(c.CastVote(members[0], p2, "hayir", 2))
    harness.run(c.CastVote(members[0], p2, "hayir", 1))          # sesi yine de sayılır
    ...                                                          # diğerleri oylar, p1 kapanır
    assert harness.uow.proposals.quadratic_extra_spent(members[0]) == 0
```

13.1 Fake'ler neden yeterince "gerçek"?

Fake repository'ler soyut sözleşmenin yalnızca soyut metotlarını gerçekler, ortak davranış (örneğin `quadratic_extra_spent`'in tanımı, `list_open`) soyut sınıftan gelir. SQLAlchemy tarafı performans için bazılarını SQL ile yeniden yazar. Bu iki gerçeklemenin aynı sonucu verdiği ayrı entegrasyon testleriyle denetlenir (`test_sql_quadratic_spend_matches_the_abstract_definition`, `test_frozen_electorate_and_ballots_survive_reloads`). Fake ile gerçeğin ayrışması, Fake kullanan test takımlarının bilinen riskidir ve bu sözleşme testleri o riski kapatır.

14 Ölçek: 3000 üyelik simülasyon

Birkaç kullanıcıyla çalışan bir forum, oylama ve ontoloji kısıtlarının gerçekte nasıl davrandığını göstermez. İlk taslağın ekran görüntüleri "az insan kullanıyor gibi" duruyordu. Bu yüzden demo verisi, 40 günlük bir topluluk hayatını canlandıran ayrık olay simülasyonuyla üretilir. Simülasyon bir öncelik kuyruğu (`heapq`) üzerinde zamanı ilerletir ve her eylemi gerçek bir komut olarak `AuthorizationProxy`'den gönderir. Demo verisi de bütün iş kurallarından geçmiştir. Reddedilen bir eylem (örneğin oylama erken kapandığı için geç kalan oy) simülasyonu durdurmaz.

- **Üyeler:** Siyasi yelpazenin uçlarından merkeze kadar yazan 45 "sesli" üye ve eğilim kovalarına dağıtılmış sessiz çoğunluk; toplam 3 000 üye ve 3 YZ ajanı. Kişisel bilgiler gerçekçi ama uydurmadır.
- **İçerik:** 25 konu, 174 mesaj, 43 teklif (açılan, reddedilen, gizlenen, uzlaşıyla kapanan konular, yönetmelik değişiklikleri, uzmanlık tanımaları).
- **Hacim:** ≈9 700 oy ve ≈13 000 defter kaydı. Kurulum ≈50 saniye sürer. Veri önce bellek içi SQLite'ta kurulur, sonra tek seferde dosyaya yedeklenir.

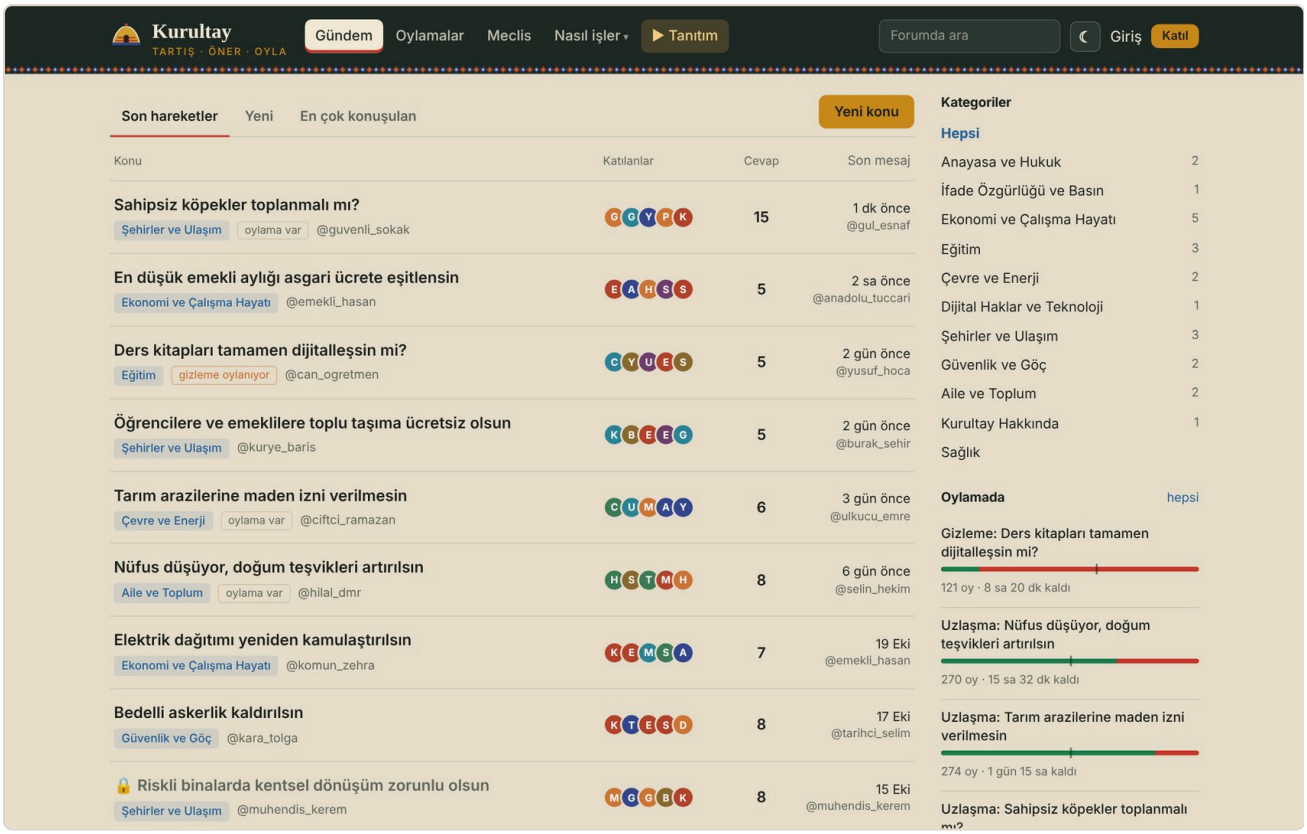
14.1 Ölçek getirdiği sorunlar ve çözümler

GÖZLENEN SORUN	NEDEN	ÇÖZÜM
Bir oy ≈100 ms sürüyordu	Her oyda bütün kullanıcılar yükleniyor ve commit'te hepsi yeniden yazılıyordu	Kimlik haritası ve kirli izleme (yalnızca değişen satır yazılır); kullanıcılar yalnızca kütük donmamışsa yüklenir
Açık oylamalar her istekte binlerce oyu okuyordu	Oylar ve kütükler her UoW'de baştan okunuyordu	Değişmez kayıtlar için `AppendOnlyReadCache`: yalnızca son okunandan sonraki satırlar (rowid) okunur
Oylama açılışında 2 000 bildirim	Bildirimler tek tek ekleniyordu	`add_many` ile toplu ekleme
Sayımda kütük her seferinde taranıyordu	Olası en yüksek ağırlık her oyda yeniden hesaplanıyordu	Kütük değişmez olduğu için strateji başına bir kez hesaplanıp saklanır
Defter sayfası yavaştı	13 000 kaydın dört kopyası her istekte doğrulanıyordu	Sonuç, defter ve düğüm imzaları değişene kadar yeniden kullanılır

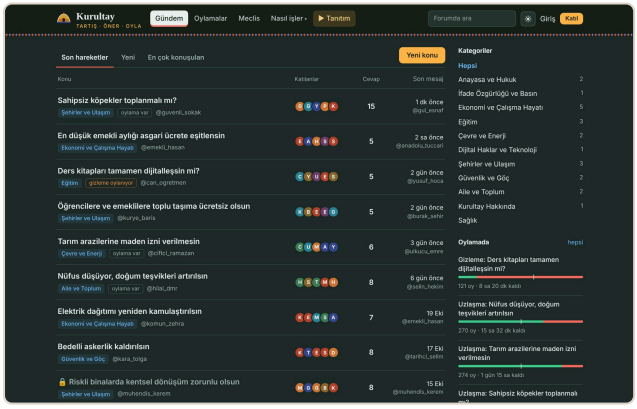
Sonuçta bir oyun Proxy, yetki zinciri, sayım, commit ve defter kaydı dahil uçtan uca işlenmesi, 1 013 kişilik kütüğü olan bir oylamada medyan **4,6 ms**, 95. yüzdelikte 7,3 ms sürer (300 oyluk ölçüm). Bu iyileştirmelerin hiçbiri alan modeline dokunmamıştır. Hepsi repository ve servis katmanında kalmış, testlerin tamamı değişmeden geçmiştir. Katmanlı mimarinin pratikteki getirisi tam olarak budur.

15 Arayüz

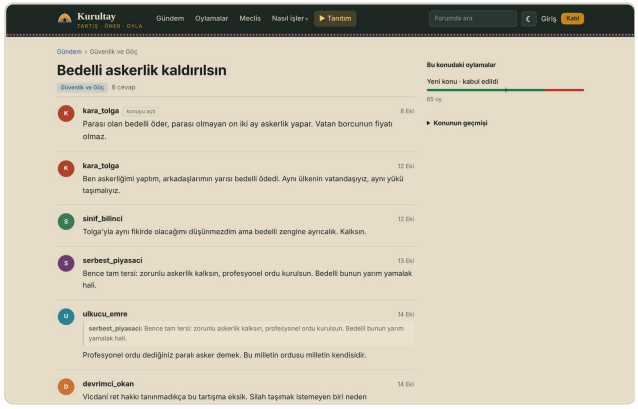
Arayüz sunucu tarafında Jinja2 ile üretilir ve iş kuralı içermez. Her sayfa bir `views` fonksiyonundan düz veri alır (CQRS okuma tarafı), her form bir komuta dönüşür. Tasarım dili ilk sürümden korunmuştur: otağ simgesi, kilim şeridi, krem renkli açık tema ve yumuşak koyu tema, her üyeye takma adından üretilen bir tamga. İkinci sürümde ana sayfa, bir tanıtım panosundan alışılmış bir forum düzenine çevrilmiştir: konu tablosu, katılanlar, cevap sayısı, son mesaj, kategoriler ve süren oylamalar.



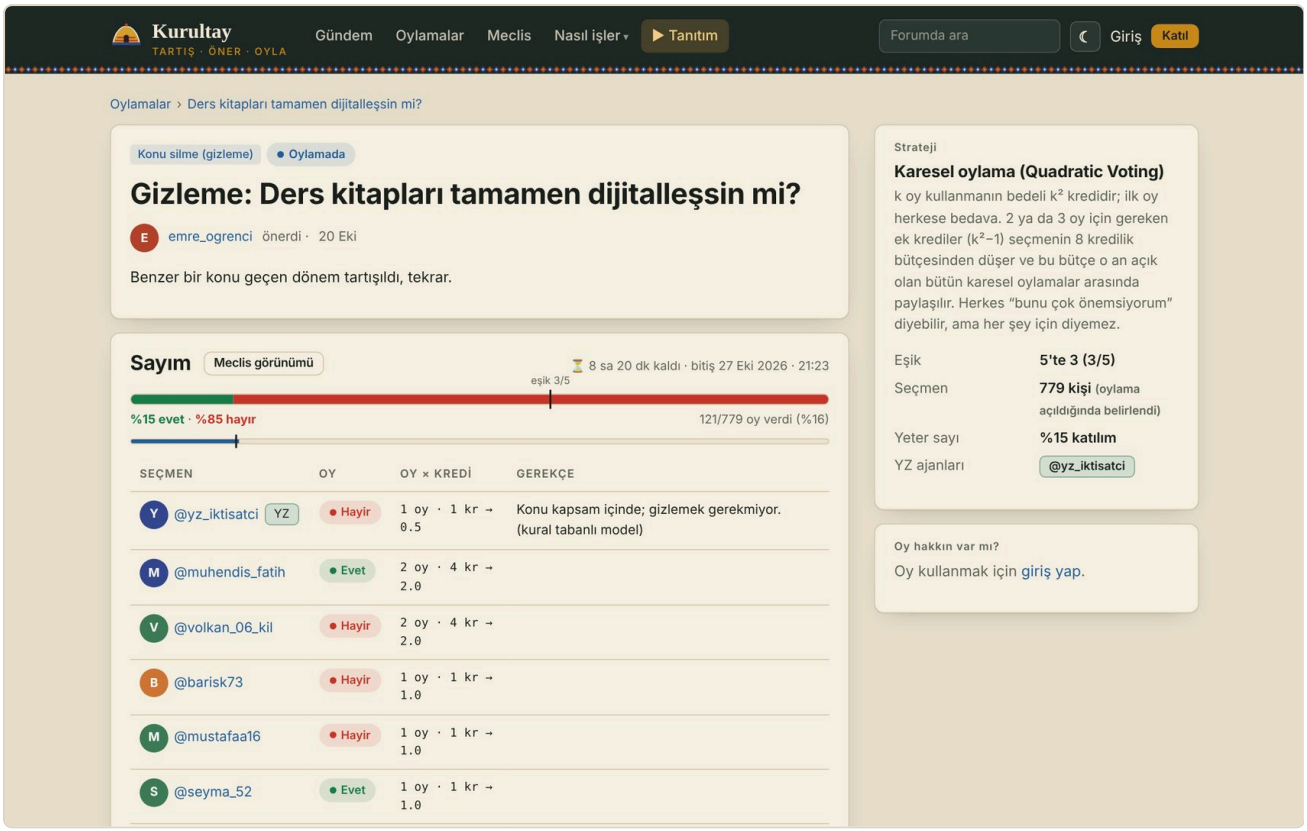
Şekil 17. Gündem (açık tema). Konular son harekete göre sıralanır. Açılması ya da gizlenmesi oylanan konular etiketle işaretlenir, uzlaşıyla kapanan konular kilitlidir. Yan sütundaki çubuklar süren oylamaların evet/hayır dağılımını ve eşiği gösterir.



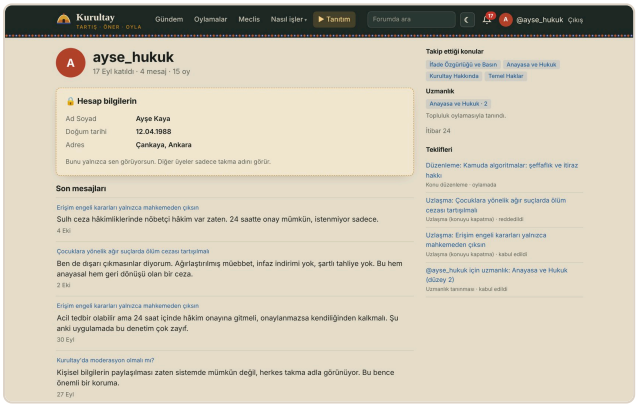
Şekil 18. Aynı sayfa, koyu tema.



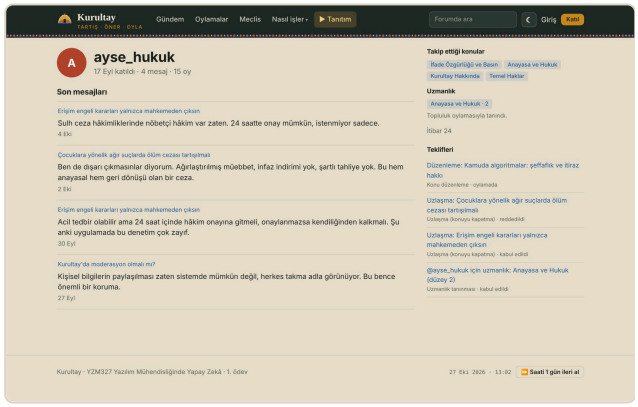
Şekil 19. Bir tartışma. Sağ sütunda konunun oylamaları ve yaşam döngüsü. Farklı görüşler yan yana durur, yalnızca tehdit engellenir.



Şekil 20. Bir teklif sayfası (karesel oylama). Sayım çubuğu, eşik işareti, katılım ve yeter sayı çubuğu, her oyun kredi bedeli ve ağırlığı, YZ ajanının gerekçesi. Sağda stratejinin açıklaması ve dondurulmuş kütüğün büyüklüğü.



Şekil 21. Hesap sahibinin kendi profili: ad, doğum tarihi ve adres yalnızca ona gösterilir.



Şekil 22. Aynı profil, başka birinin gözünden: yalnızca takma ad, ilgi alanları ve katkılar.

16 Ön-otopsi, sınırlılıklar ve sonraki adımlar

Ders notundaki ön-otopsi (Klein, 2007) yöntemi uygulanmıştır: "Aradan altı ay geçti ve Kurultay tamamen başarısız oldu. Neden?" Olası nedenler olasılık ve etkiye göre sıralanmış, sağ üst köşedekiler için bugünden önlem yazılmıştır.

	BAŞARISIZLIK SENARYOSU	OLASILIK	ETKİ	BUGÜNKÜ DURUM VE ÖNLEM
A	Sahte hesaplar (Sybil) karesel oylamanın kredi bütçesini çoğalttı	Yüksek	Yüksek	Bugün kimlik doğrulanmıyor; karesel oylamanın bilinen zaafıdır. Önlem: doğrulanmış kimlik (telefon ya da e-Devlet) ve kütüğe girmek için en az üyelik süresi. Kütüğün açılışta dondurulması, oylama başladıktan sonra açılan hesapları zaten dışarıda bırakıyor.
B	Tehdit kuralı yeni kalıpları kaçırdı, ciddi bir tehdit yayında kaldı	Orta	Yüksek	İkinci savunma hattı var: gizleme oylaması. Önlem: ikinci etiketleyiciyle seti büyütmek, κ ölçmek, geliştirme ve test setini ayırmak (Bölüm 11).
C	Katılım düştü, oylamalar yeter sayıya ulaşamadı	Orta	Orta	Yeter sayı oylanabilir (demoda %15'e indi); kütükteki herkese bildirim gider; erken karar uzun beklemeyi önler.
D	YZ ajanı teklif metnindeki bir istem enjeksiyonuyla yönlendirildi	Orta	Düşük	Ajan oyu yarım sayılır, teklif açamaz, yanıt katı biçimde ayrıştırılır, ayrıştırılamazsa kurala düşer, yönetmelikle tamamen kapatılabilir.
E	Defter ile veritabanı ayrıştı (çökme)	Düşük	Orta	Defter kaydı, iş değişikliği commit edildikten sonra ayrı bir Unit of Work'te yazılıyor. Arada çöken bir süreç, değişikliği defterde iz bırakmadan kaydedebilir. Önlem: olayların aynı işlemde bir *outbox* tablosuna yazılması.
F	Kural tabanlı çözümler yeterli çıktı, BDM hiç gerekmedi	Yüksek	Düşük	Bu bir başarısızlık değil, Hafta 2'nin beklediği sonuçtur. Kural, ileride gelecek bir modelin yenmesi gereken çizgidir.

16.1 Bilinen sınırlılıklar

- **Tek süreç, tek veritabanı.** Message Bus bir kilitle aynı anda tek mesaj zinciri işler ve SQLite kullanılır. Ders ödevi ve birkaç bin üye için yeterlidir, yatay ölçeklenmez. Repository ve UoW soyutlamaları sayesinde PostgreSQL'e geçiş adapters katmanıyla sınırlı kalır.
- **Defter bir öğretim modelidir.** Düğümler aynı makinededir ve uzlaşma salt çoğunluktur. Gerçek bir dağıtık defter, ayrı makineler ve Bizans hatalarına dayanıklı bir uzlaşma protokolü gerektirir. Defter denetim kaydıdır, durum ondan yeniden kurulmaz (tam olay kaynaklama yok).
- **Ontoloji anahtar kelimeye dayanır.** Türkçe çekim ekleri kelime başı eşleşmesiyle kısmen karşılanır, eşanlamlılar karşılanmaz. Ontolojiye yeni kavram eklemek için ayrı bir teklif türü yoktur, ilk ağaç başlangıç verisidir.
- **Kütük dondurmanın bedeli.** Oylama sırasında katılan yeni bir üye o oylamaya katılamaz. Bu, akın saldırısına karşı bilinçli olarak ödenmiş bir bedeldir ve oylama süresi kısa tutularak (varsayılan 3 gün) hafifletilir.
- **Ölçüm küçük ve tek kişiliktir.** 60 cümle ve 25 konu, yöntemler arasındaki farkı göstermeye yeter, ancak genelleme iddiası için yetmez.

17 Sonuç

Kurultay'ın ikinci sürümü, ilk sürümün fikrini korurken onu ders kurallarının ölçülebildiği bir yapıya taşır. Alan modeli altyapıdan habersizdir ve bu bir testle güvence altındadır. Her durum değişikliği bir komutla başlar, tek bir Message Bus'tan geçer, olaylarla yayılır ve silinemez bir deftere yazılır. İstenen sekiz GoF deseninin her biri adlandırılmış bir soruna yanıttır ve her biri için seçilmeyen alternatifin gerekçesi verilmiştir. Hiçbir soyutlama tek gerçeklemeli değildir.

Raporun en önemli katkısı bir desen listesi değil, üç düzeltmedir. Bunlar ilk taslağa bir değerlendiricinin gözüyle bakılınca görüldü. Karesel oylama, kredi kıt ve paylaşılan hale getirilerek gerçekten azınlığı koruyan bir yönteme dönüştürüldü ve bunun etkisi canlı veride gözlendi. Tehdit kuralı, bir temel çizgi merdiveniyle ölçüldü ve naif yöntemin hiçbir şey yapmamaktan daha pahalı olduğu gösterildi. Gizli global önbellekler enjekte edilen açık nesnelere dönüştürüldü ve bir mimari testle korundu. Her üç düzeltme de alan modeline ya hiç dokunmadan ya da tek bir fonksiyonla yapılabildi. Katmanlı mimarinin vaat ettiği değişebilirlik budur.

Sonraki adımlar ön-otopsinin sağ üst köşesindedir: kimlik doğrulamasıyla Sybil direnci, ikinci etiketleyiciyle büyütülmüş bir değerlendirme seti ve defter tutarlılığı için outbox deseni.

YZ kullanım beyanı

Bu çalışmada Anthropic'in Claude modeli kodlama ve yazım yardımcısı olarak kullanılmıştır. YZ; Python kodunun, testlerin, demo simülasyonu metinlerinin, Mermaid diyagram kaynaklarının ve bu raporun taslaklarının üretilmesinde ve kodun ders kurallarına göre gözden geçirilmesinde rol almıştır. Öğrenci; ödev isterlerini ve ders notlarını sağlamış, ürünün yönünü belirlemiştir (fikir özgürlüğü ilkesi, siyasi yelpaze, forum görünümü, tema). Uygulamayı kendi bilgisayarında çalıştırıp sınamış, hataları bildirmiş ve raporu gözden geçirmiştir. Rapordaki bütün sayılar depodaki komutlarla üretilmiştir ve Ek A'daki adımlarla yeniden üretilebilir. YZ çıktılarının doğruluğu 149 otomatik testle ve `mypy` ile denetlenmiştir. Çalışma boyunca ücretli bir servis ya da dış API kullanılmamıştır.

Kaynakça

1. Percival, H., Gregory, B. (2020). *Architecture Patterns with Python* (“Cosmic Python”). O'Reilly.
2. Gamma, E., Helm, R., Johnson, R., Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
3. Martin, R. C. (2017). *Clean Architecture*. Prentice Hall.
4. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
5. Ramalho, L. (2022). *Fluent Python* (2. baskı), Bölüm 10: Design Patterns with First-Class Functions. O'Reilly.
6. Huyen, C. (2022). *Designing Machine Learning Systems*, Bölüm 1–2. O'Reilly.
7. Sculley, D. vd. (2015). Hidden Technical Debt in Machine Learning Systems. *NeurIPS* 28.
8. Klein, G. (2007). Performing a Project Premortem. *Harvard Business Review*, 85(9).
9. Dorard, L. (2015). *The Machine Learning Canvas*. ownml.co.
10. Lalley, S. P., Weyl, E. G. (2018). Quadratic Voting: How Mechanism Design Can Radicalize Democracy. *AEA Papers and Proceedings*, 108, 33–37.
11. Posner, E. A., Weyl, E. G. (2018). *Radical Markets*, Bölüm 2. Princeton University Press.
12. Haber, S., Stornetta, W. S. (1991). How to Time-Stamp a Digital Document. *Journal of Cryptology*, 3(2), 99–111.
13. Cohen, J. (1960). A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement*, 20(1), 37–46.
14. *Virginia v. Black*, 538 U.S. 343 (2003): “true threat” tanımı.
15. Takan, S. (2026). *Tasarım Desenleri* ders notları. YZM327, Ankara Üniversitesi.
16. Takan, S. (2026). *Hafta 2: Problem Çerçeveleme, Başarı Metrikleri ve Temel Çizgi* ders notları. YZM327, Ankara Üniversitesi.

A Ek A: Kurulum, çalıştırma ve yeniden üretim

Uygulama Windows'ta tek tıkla açılacak şekilde paketlenmiştir. Gömülü Python ile birlikte gelir ve internet ya da ücretli servis gerektirmez.

NE YAPILACAK?	NASIL?
Uygulamayı açmak	Masaüstündeki **Kurultay** kısayolu ya da proje klasöründeki `Kurultay.cmd`. İlk açılışta demo verisi kurulur (≈1 dk), sonra uygulama varsayılan tarayıcıda kendi penceresinde açılır.
Demo verisini baştan kurmak	`Kurultay (demo verisini yenile).cmd`
Giriş yapmak	Giriş sayfasında listelenen herhangi bir demo üyesi; şifre `demo1234`. Yeni üye de kaydedilebilir.
Testleri çalıştırmak	`Testleri calistir.cmd` ya da `python -m pytest` (149 test, ≈32 sn). Özet `docs/test-ozeti.json`'a yazılır.
Ölçümü yeniden üretmek	`python -m kurultay.evaluation` → Bölüm 11'deki tablolar ve `docs/olcum.json`
Tip denetimi	`python -m mypy src`
Diyagramları yeniden çizmek	`mmdc -i docs/diyagramlar/NN-ad.mmd -o NN-ad.svg -b white` (Mermaid CLI)
Yerel dil modelini açmak (isteğe bağlı)	`ollama pull llama3.2` ve Ollama'yı çalıştırmak. `@yz_teknolog` ajanı otomatik olarak kullanır. Adres `OLLAMA_URL` ile değiştirilebilir. Ollama yoksa ajan kural tabanlı modele düşer.
Bir oylamayı canlı izlemek	Üst menüde **Meclis** → bir senaryo → "Canlı başlat"
Defterin kurcalanmasını görmek	**Nasıl işler → Defter** → "Bir düğümü kurcala", ardından "Çoğunluktan onar"

Klasör yapısı

```
ödev1-kurultay/
├─ src/kurultay/
│  ├─ domain/          alan modeli: aggregate'ler, değer nesneleri, desenler, ontoloji, defter
│  ├─ service_layer/   işleyiciler, Message Bus, UoW, Observer, proxy'ler, YZ ajanı, görünümler
│  ├─ adapters/        SQLAlchemy şeması ve repository'ler, YZ adaptörleri, defter düğümleri
│  ├─ entrypoints/     web uygulaması (FastAPI + Jinja2), demo simülasyonu, canlı gösterim
│  ├─ evaluation/      etiketli set, metrikler, temel çizgi merdiveni
│  └─ bootstrap.py     bileşim kökü
├─ tests/              unit/ · integration/ · e2e/ · fakes.py
├─ docs/diyagramlar/   14 Mermaid kaynağı ve SVG çıktıları
├─ araclar/            Windows başlatıcısı
└─ RAPOR.md · README.md
```